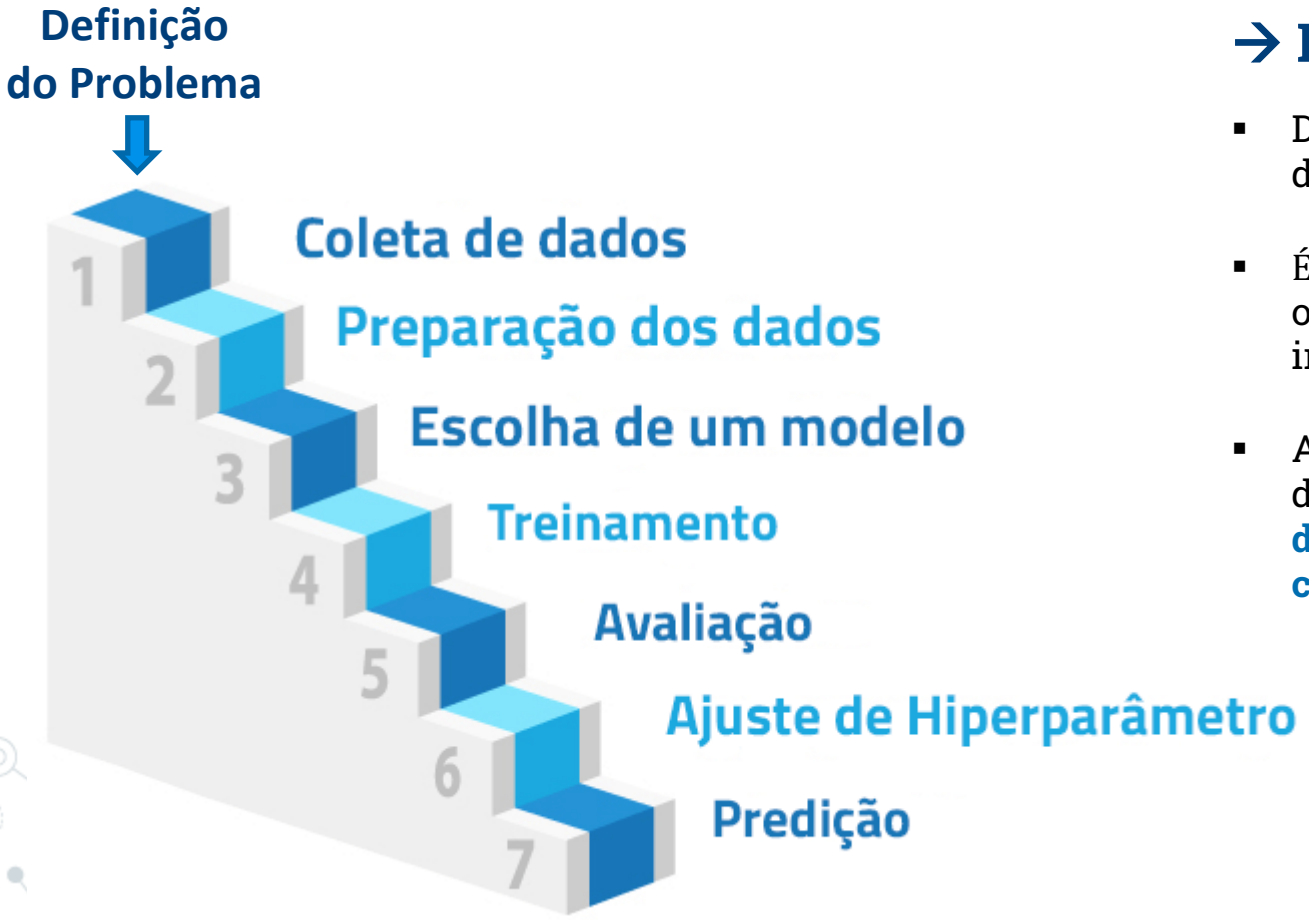
Inatel

Inteligência Artificial

Profª. Victoria Dala Pegorara Souto

Machine Learning

Etapas do Projeto de ML

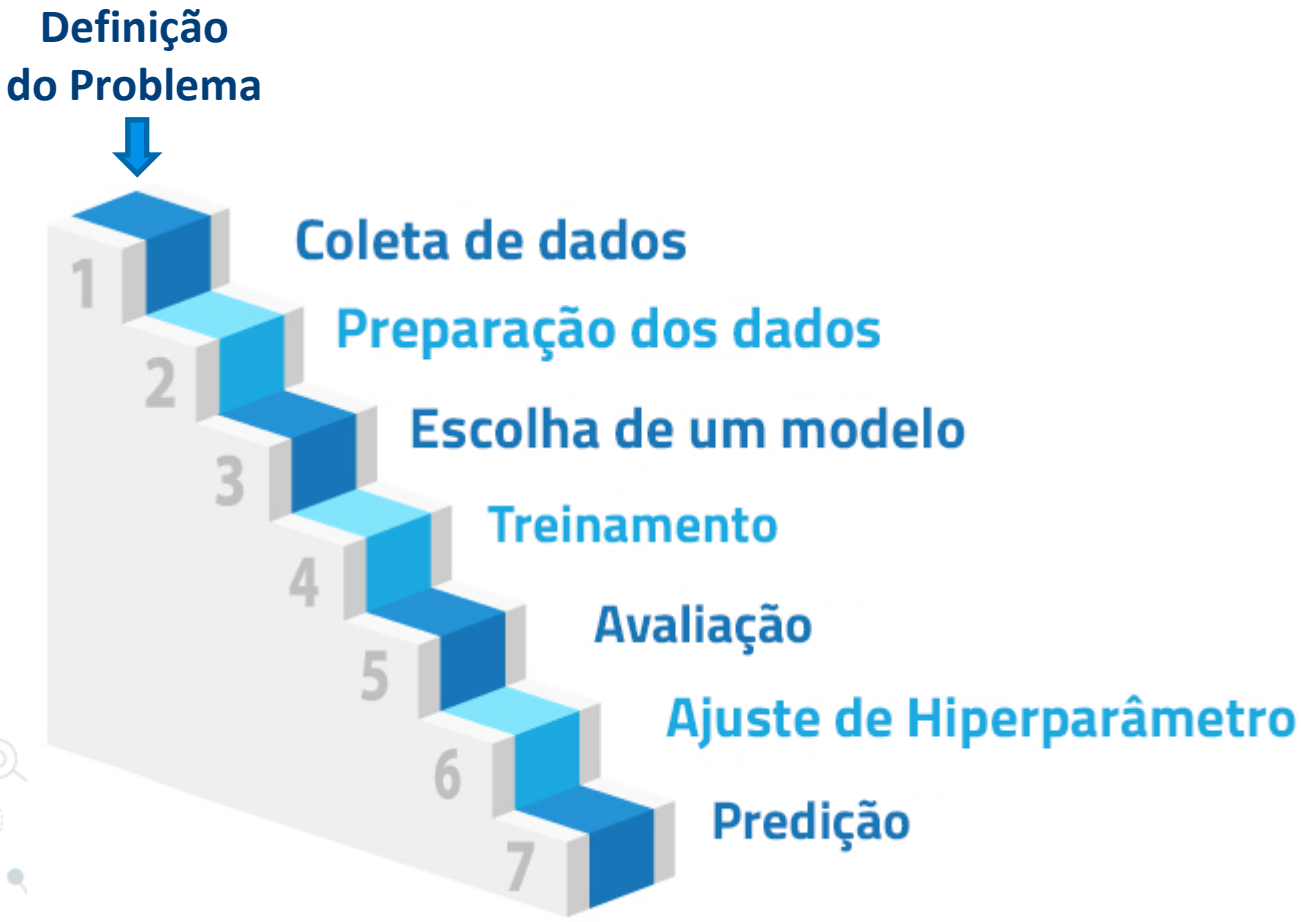


→ Definição do problema:

- Definição e entendimento do problema a ser abordado e do objetivo a ser atingido.
- É importante saber de forma clara e bem definida o objetivo para evitar que o projeto tenha resultados inesperados ou ineficientes.
- Após definido o objetivo do problema, o projetista irá definir **como o problema será abordado, qual abordagem do ML será utilizada e qual medida de desempenho será considerada.**

Machine Learning

Etapas do Projeto de ML

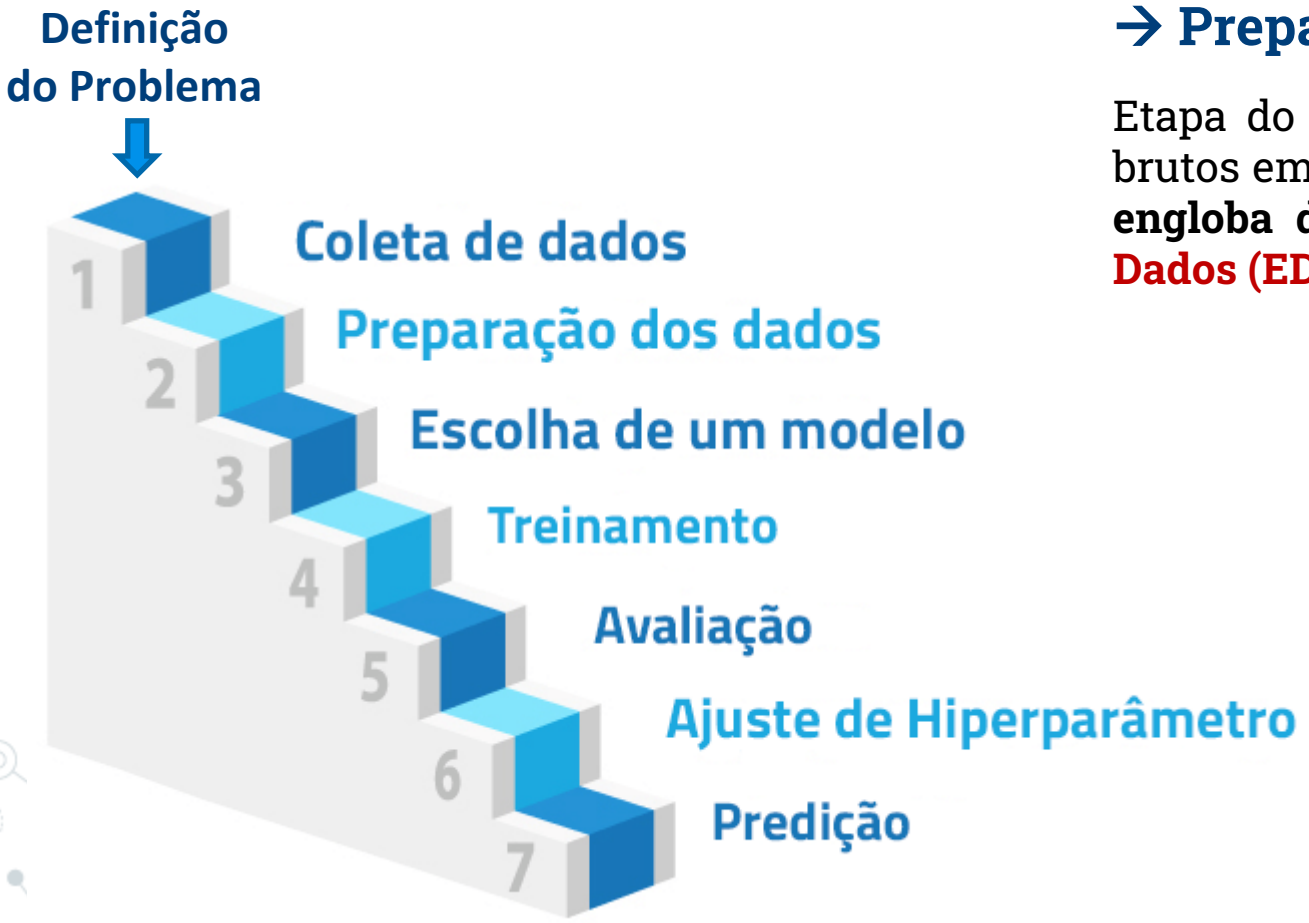


→ Coleta de Dados:

- É realizada visando atender os requisitos de volume e qualidade dos dados coletados.
- Dependendo do objetivo do projeto, não é necessariamente interessante um alto volume de dados, mas sim dados de maior qualidade e representatividade.
- Após realizar a coleta dos dados é importante realizar uma rápida **análise dos dados** visando verificar o tipo dos dados coletados, número de atributos e seu significado para o problema a ser abordado.
- Além disso, os dados devem ser **separados em conjunto de treinamento e teste**. A partir desse momento, qualquer visualização e processamento dos dados deve ser realizado apenas no conjunto de treinamento.

Machine Learning

Etapas do Projeto de ML



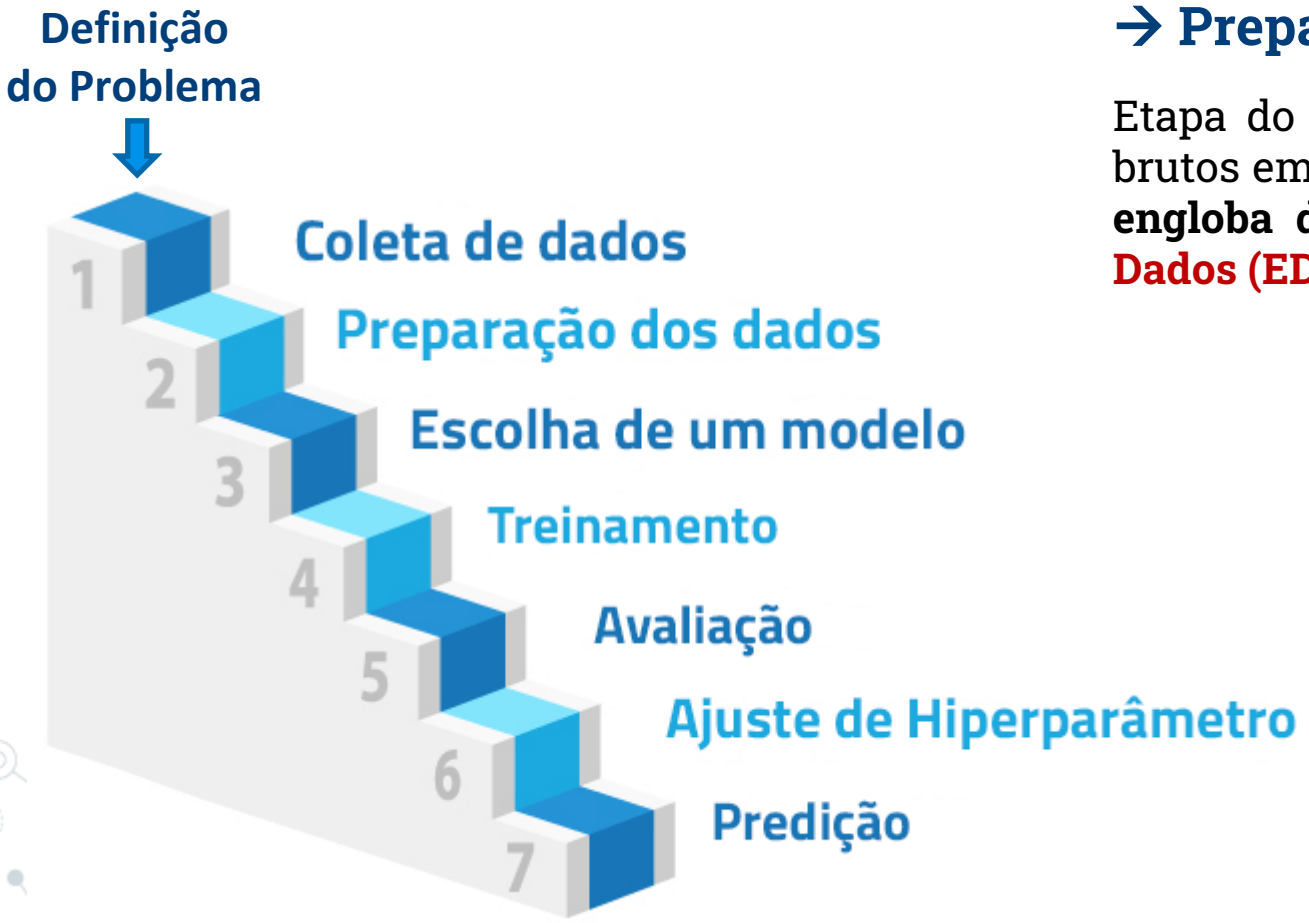
→ Preparação dos Dados:

Etapa do projeto de ML responsável por transformar os dados brutos em um conjunto adequado para a modelagem. Essa etapa **engloba duas fases complementares: Análise Exploratória de Dados (EDA) e Pré-processamento de Dados**.

- **EDA:** Busca-se compreender os dados, analisando distribuições, relações entre variáveis, presença de valores ausentes, outliers, inconsistências e possíveis vieses. Essa análise orienta decisões importantes sobre quais transformações e ajustes serão necessários.

Machine Learning

Etapas do Projeto de ML



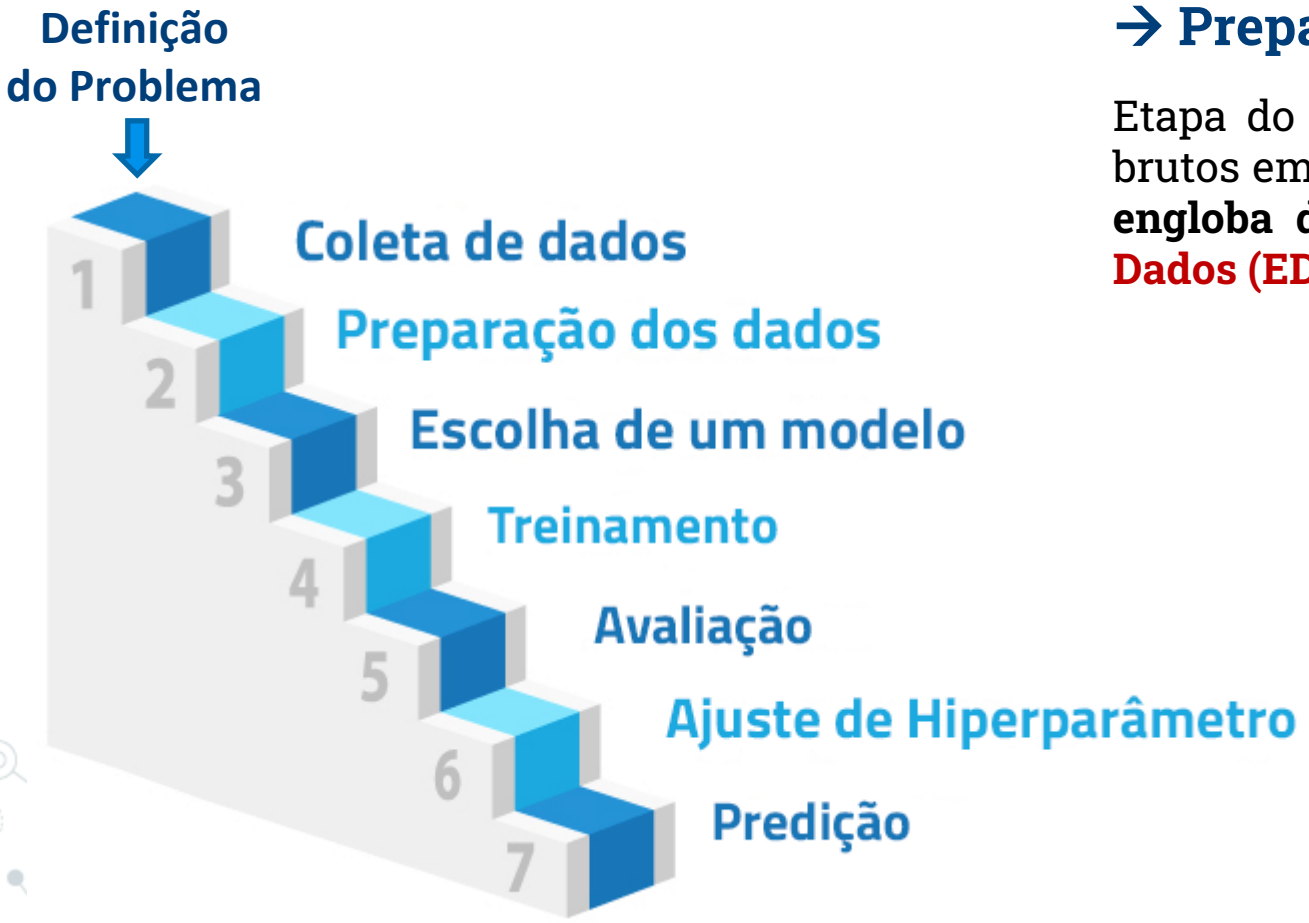
→ Preparação dos Dados:

Etapa do projeto de ML responsável por transformar os dados brutos em um conjunto adequado para a modelagem. Essa etapa **engloba duas fases complementares: Análise Exploratória de Dados (EDA) e Pré-processamento de Dados**.

- **Pré-processamento de Dados**: Os dados são efetivamente tratados e transformados, incluindo limpeza, tratamento de valores faltantes, normalização ou padronização, codificação de variáveis categóricas, engenharia e seleção de atributos, além da divisão dos dados em conjuntos de treino, validação e teste.

Machine Learning

Etapas do Projeto de ML



→ Preparação dos Dados:

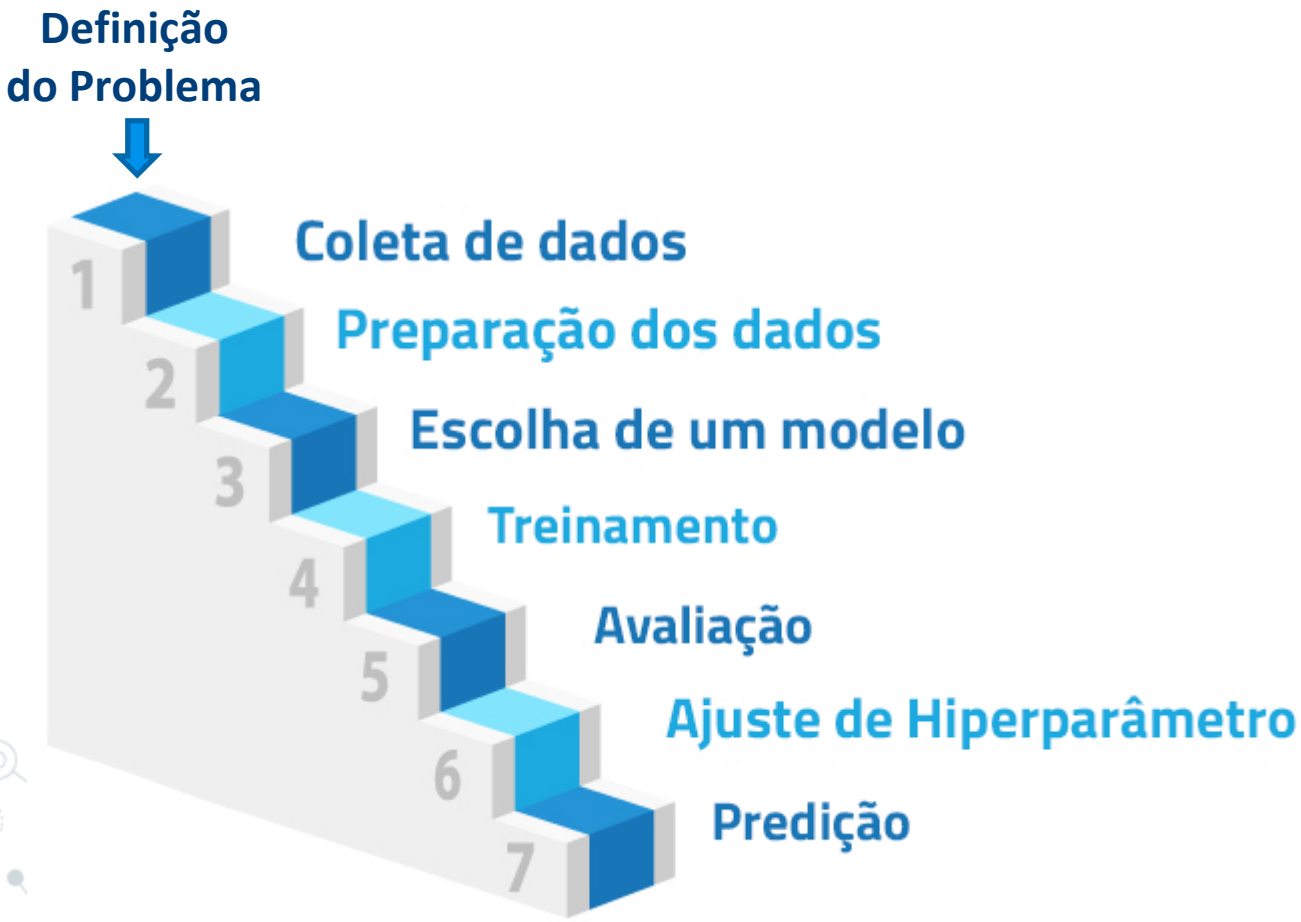
Etapa do projeto de ML responsável por transformar os dados brutos em um conjunto adequado para a modelagem. Essa etapa **engloba duas fases complementares: Análise Exploratória de Dados (EDA) e Pré-processamento de Dados**.

Objetivo Final:

Garantir dados consistentes, bem compreendidos e adequadamente estruturados, possibilitando um treinamento mais eficiente e resultados mais confiáveis do modelo.

Machine Learning

Etapas do Projeto de ML



→ Escolha de um Modelo:

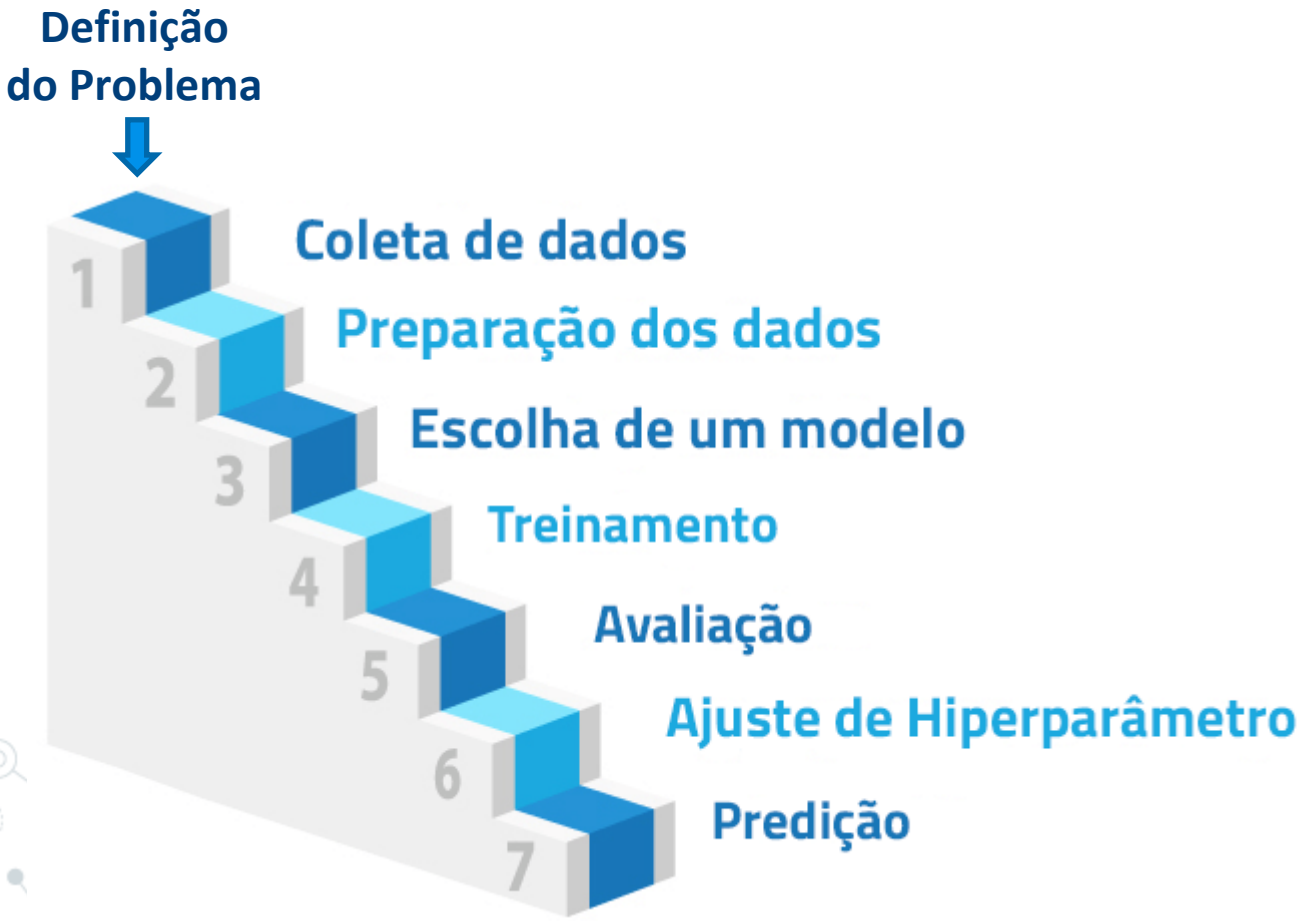
- Com os dados devidamente pré-processados e agrupados em conjunto de teste e treinamento, nesta etapa, é **escolhido o modelo de ML a ser utilizado conforme o tipo de dado fornecido e o objetivo do problema previamente definido.**

→ Treinamento:

- Após escolhido o modelo de ML, realiza-se o treinamento do modelo escolhido, que uma vez treinado estará pronto para fazer a predição/regressão das novas informações que serão fornecidas ao sistema.

Machine Learning

Etapas do Projeto de ML



→ Avaliação:

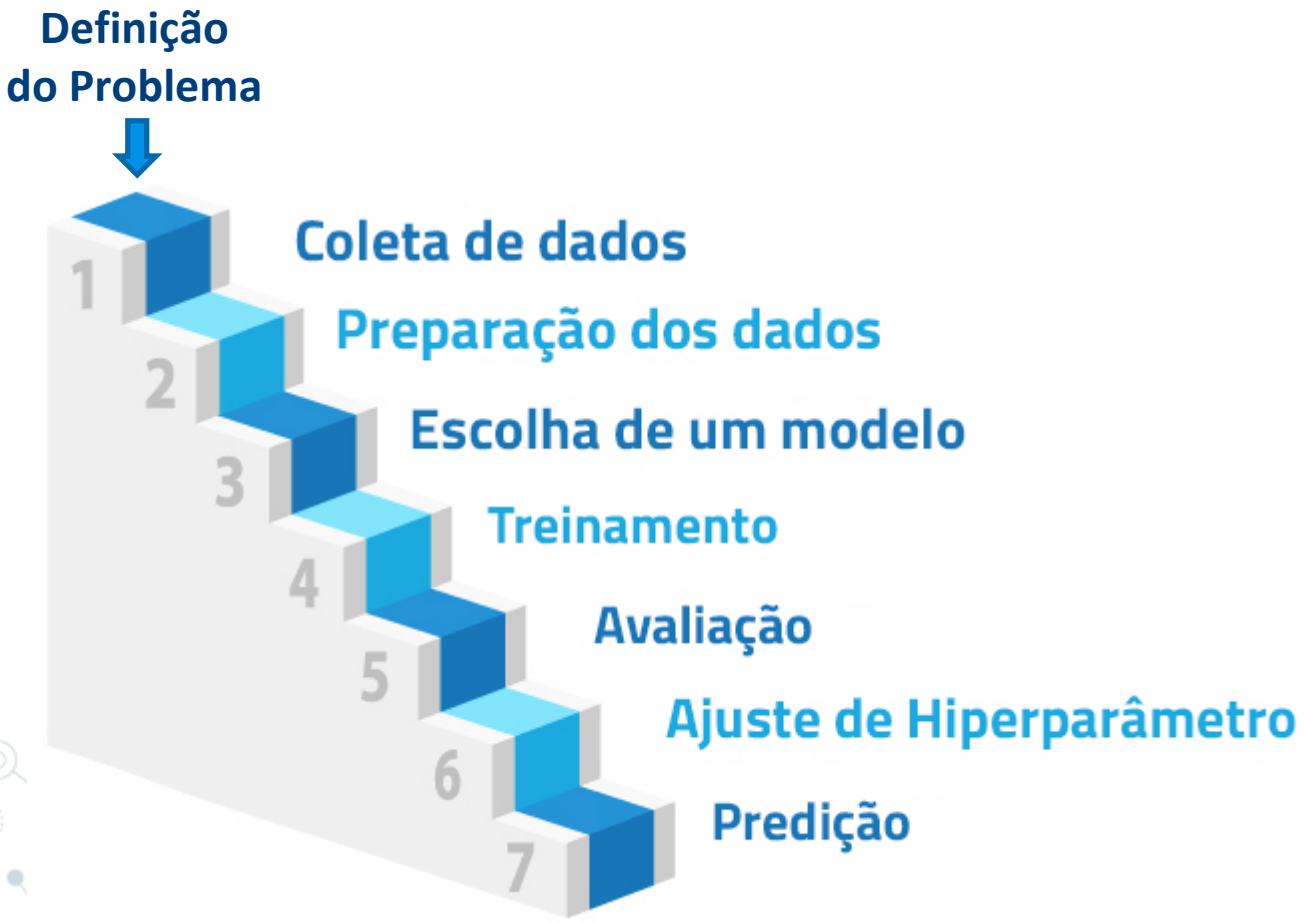
- Ao fim do treinamento, é feita a avaliação do modelo escolhido a partir do conjunto de dados de teste. **A avaliação do modelo é realizada considerando a métrica de desempenho previamente definida.**

→ Ajuste de Hiperparâmetros:

- Nesta etapa o modelo escolhido na etapa anterior é otimizado a partir da otimização dos seus hiperparâmetros.
- **Busca-se a melhor combinação de parâmetros que otimizem o desempenho do modelo proposto**.

Machine Learning

Etapas do Projeto de ML

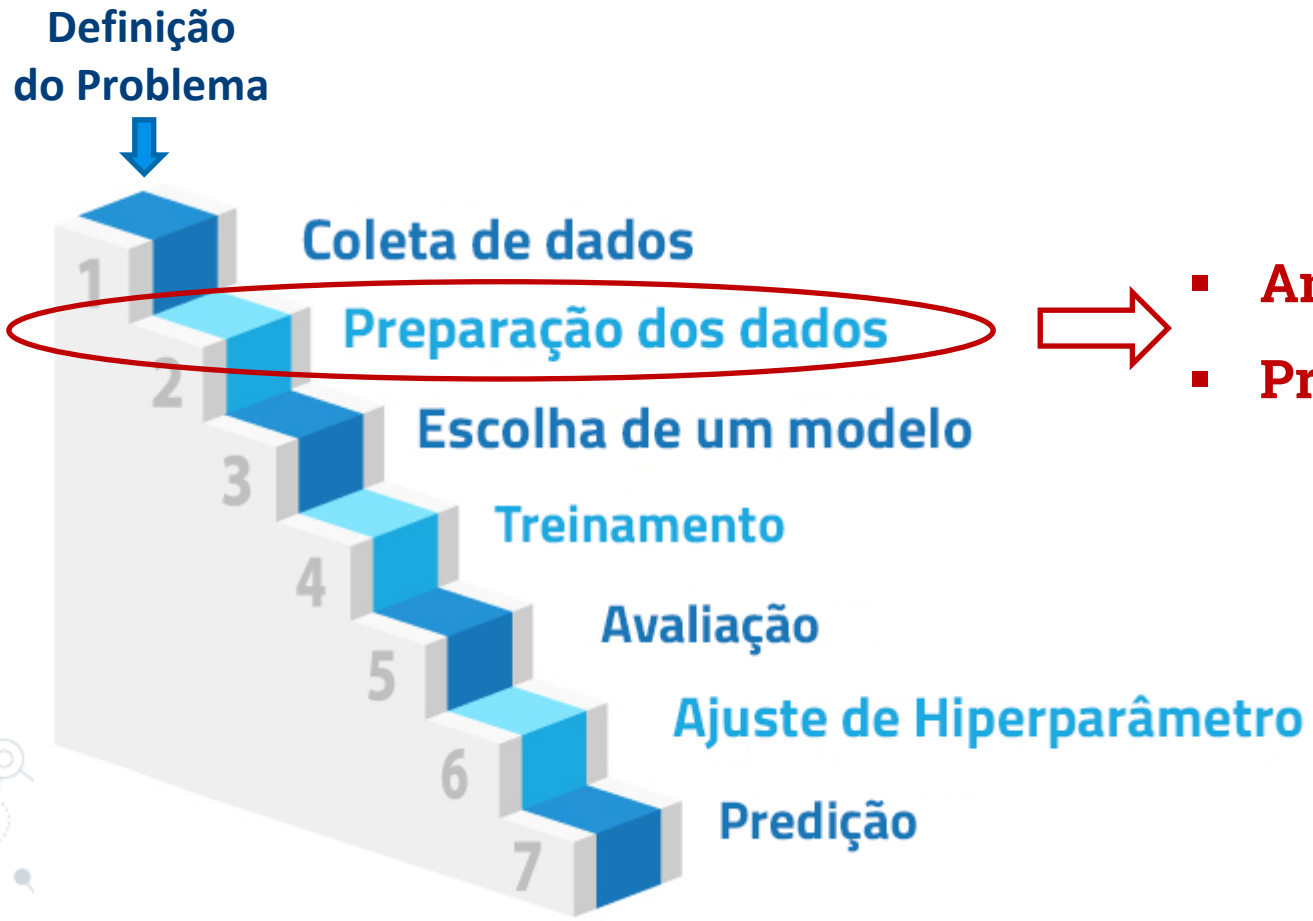


→ Predição:

- Etapa de implantação da solução proposta. Nesta etapa o modelo proposto será colocado em produção.
- Vale ressaltar que mesmo após colocado em produção, o modelo deve ser continuamente monitorado e as devidas **manutenções** devem ser realizadas para garantir o seu desempenho.

Machine Learning

Etapas do Projeto de ML



- **Análise Exploratória de Dados (EDA)**
- **Pré-processamento dos Dados.**

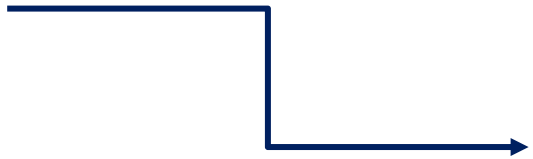
Análise Exploratória de Dados (EDA)

- Etapa executada antes da construção e treinamento dos modelos de ML.
- É o primeiro passo em qualquer projeto de ciência de dados, incluindo ML.

EDA nos ajudar a identificar:

- ✓ Estrutura e **QUALIDADE** dos Dados
- ✓ Distribuições e Tendências
- ✓ Relações e Correlações
- ✓ Insights e Hipóteses para Modelagem

A EDA transforma dados crus em **conhecimento visual e estatístico**, garantindo que você não gaste tempo aplicando modelos sofisticados em dados que estão "sujos" ou que não fazem sentido.



- ✓ Erros óbvios nos dados
- ✓ Valores faltantes, Duplicados, Outliers, Tipagem de variáveis.
- ✓ Compreender os padrões nos dados.
- ✓ Encontrar relações entre as variáveis.
- ✓ Descobrir quais variáveis realmente importam.

Análise Exploratória de Dados (EDA)

Por que realizar análise exploratória?

- A análise exploratória é uma espécie de curadoria dos dados.
- Ela garante que o modelo de ML não aprenda com lixo.
- Se o dado de entrada é ruim, a previsão será pior ainda.

Aspecto	Sem Análise Exploratória	Com Análise Exploratória
Confiança	Alta chance de resultados enganosos.	Resultados robustos e explicáveis.
Custo	Desperdício de tempo refinando modelos ruins.	Foco direto nas variáveis que importam.
Conhecimento	Visão superficial dos dados.	Domínio profundo sobre o fenômeno estudado.

A EDA não é um processo linear. Você frequentemente voltará a ela após tentar criar um modelo e perceber que algo ainda não faz sentido. É um processo **CÍCLICO** de descoberta.

Análise Exploratória de Dados (EDA)

Inspeção Visual e Estatística

→ Inspeção Estatística

- Foco é o **rigor**.
- Resumir grandes volumes de dados em poucos indicadores numéricos que facilitam a comparação.

✓ **Validar a Variabilidade:** Através do desvio padrão e variância, você entende se seus dados são estáveis ou "selvagens".

✓ **Identificar Tendências Centrais:** Definir o que é um valor "normal" para aquele conjunto (Média vs. Mediana).

✓ **Detectar Discrepâncias:** Quantificar quantos valores estão fora da curva (outliers).





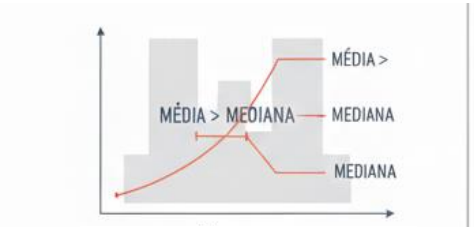
Análise Exploratória de Dados (EDA)

Inspeção Visual e Estatística

→ Inspeção Visual

- Detectar Padrões.

  - ✓ **Identificar a Distribuição:** Ver se os dados estão concentrados em um lado (assimetria) ou se seguem o formato de sino (distribuição normal).
  - ✓ **Encontrar Agrupamentos (Clusters):** Perceber se existem grupos naturais nos dados que a estatística isolada não mostraria.
  - ✓ **Confirmar Correlações:** Ver se existe uma relação clara entre duas variáveis (ex: quanto mais tempo de estudo, maior a nota).



Histograma
Forma de Distribuição

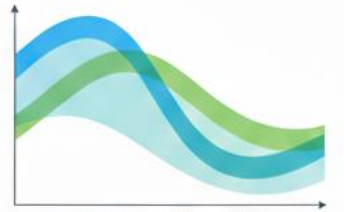
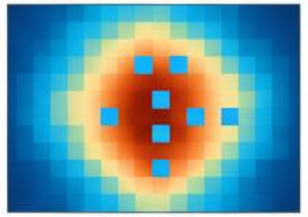
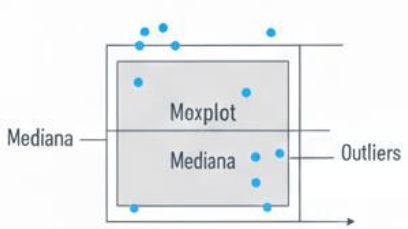


Gráfico de Densidade



Mapa de Calor



OUTLIERS

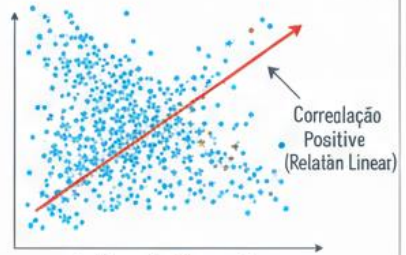
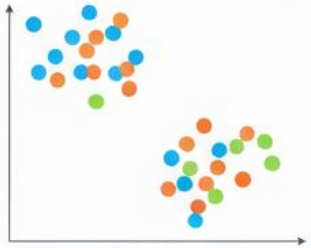


Gráfico de Dispersão



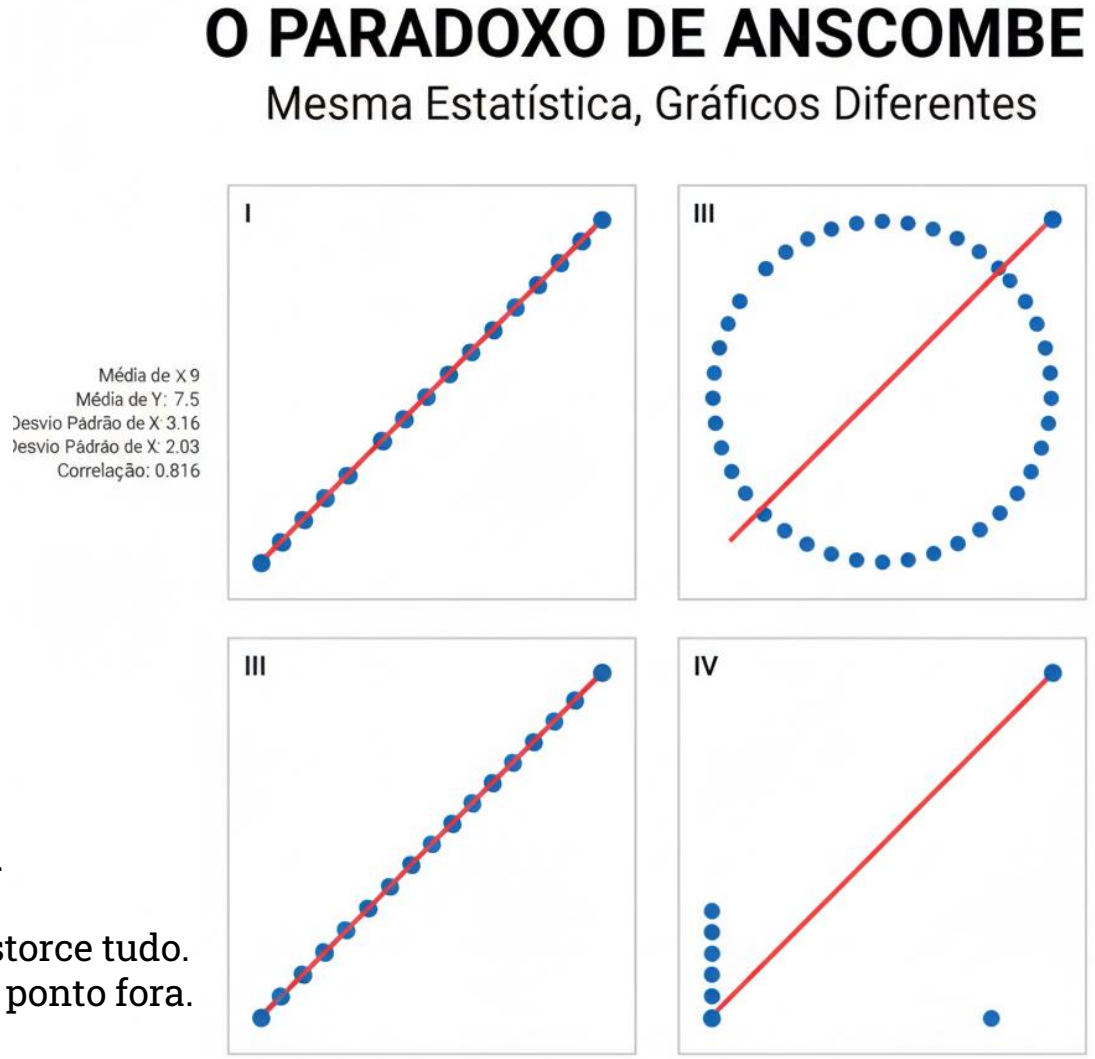
Clusters

Análise Exploratória de Dados (EDA)

Inspeção Visual e Estatística

→ **Por que utilizas as duas simultaneamente?**

Conjuntos de dados com as mesmas estatísticas (ex.: média, variância, desvio padrão), mas com comportamentos totalmente diferentes!!!



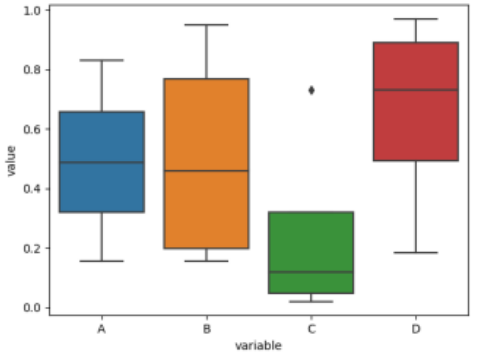
I. Relação linear simples.
II. Curva perfeita.
III. Tem um outlier que distorce tudo.
IV. Linha vertical com um ponto fora.

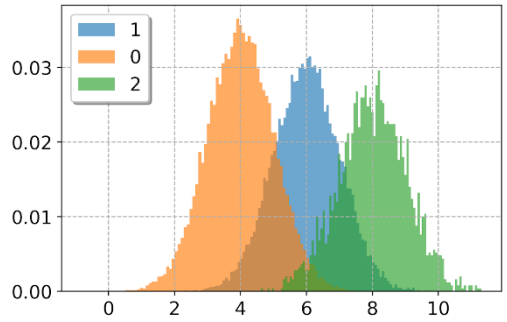
Análise Exploratória de Dados (EDA)

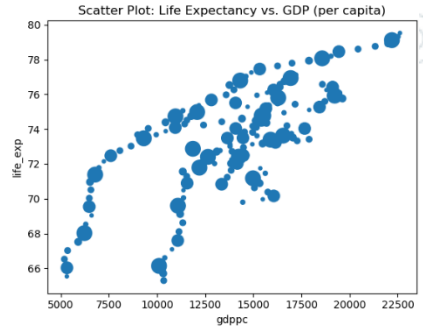
Inspeção Visual e Estatística

Utilizamos ferramentas estatísticas e de visualização como

- Histogramas
- *Heatmaps* de correlação
- *Boxplots*
- Gráficos de barras
- Diagramas de dispersão







Bibliotecas mais usadas:

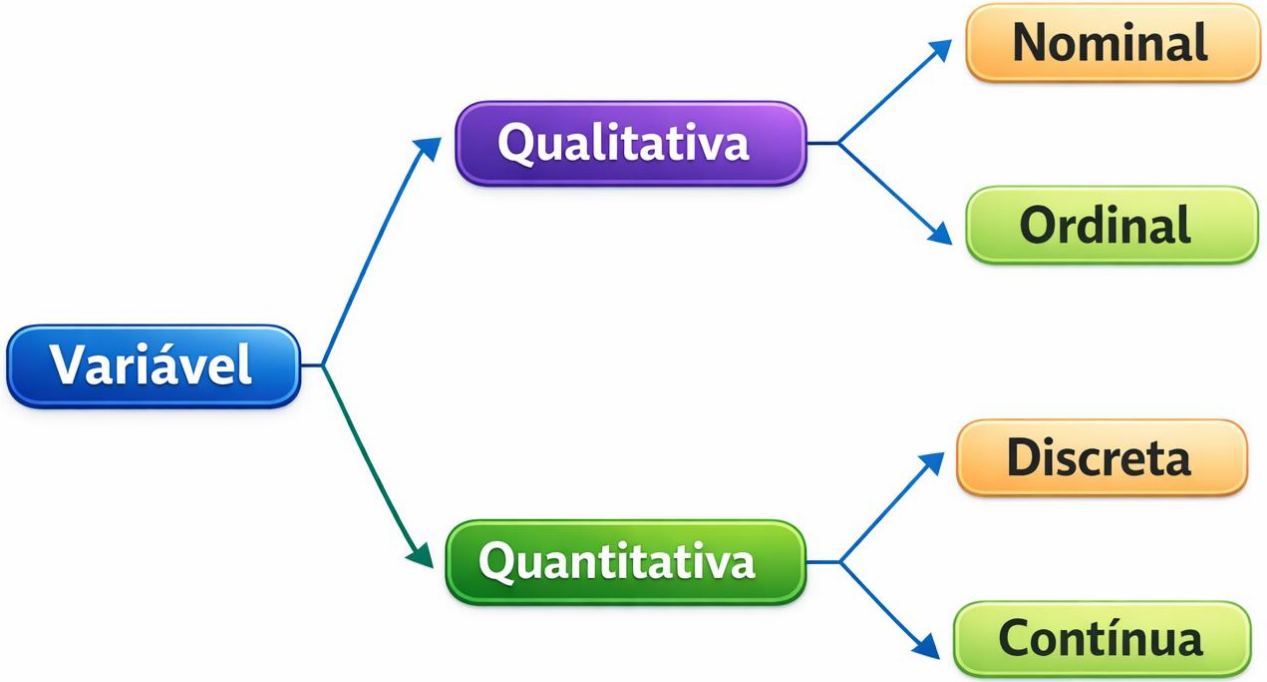
- Pandas
- Numpy
- Scikit-learn
- Matplotlib
- Seaborn
- Pandas profiling
- Polars

Análise Exploratória de Dados (EDA)

Tipos de Dados

Antes de explorarmos os dados, precisamos entender a sua taxonomia.

- As variáveis (i.e., atributos ou rótulos) podem ser classificadas da seguinte forma:



Análise Exploratória de Dados (EDA)

Tipos de Dados

→ Variáveis Qualitativas (Categóricas)

- São categorias, símbolos, nomes ou rótulos.
- Elas descrevem uma qualidade.
- Algumas podem ser ordenadas, mas operações aritméticas não são aplicáveis.



Nominal: Não existe uma ordem inerente.

- **Exemplos:** Sistemas Operacionais (Linux, Windows, macOS), Cores de LED, Tipos de Banco de Dados.

Ordinal: Existe uma hierarquia ou ordem lógica.

- **Exemplos:** Febre (baixa, média, alta), Nível de Senioridade (Junior, Pleno, Sênior), Planos de Assinatura (Free, Premium, Enterprise).

Análise Exploratória de Dados (EDA)

Tipos de Dados



→ Variáveis Quantitativas (Numéricas)

- São números reais.
- Podem ser ordenados e usados em operações aritméticas (e.g., média, variância).
- Possuem unidade de medida.

Discreta: Valores contáveis, geralmente números inteiros.

- **Exemplos:** Quantidade de núcleos da CPU, número de bugs, total de usuários ativos.

Contínua: Valores que podem assumir qualquer número dentro de um intervalo.

- **Exemplos:** Tempo de resposta de uma API, latência de rede, tamanho de um arquivo em MB, temperatura do processador.

Análise Exploratória de Dados (EDA)

Inspeção Inicial dos Dados

- Inicialmente, verificamos algumas informações básicas dos dados.
- Para tal, após carregarmos os dados, usamos atributos e métodos da biblioteca **pandas**.

→ **Dimensão dos Dados (linhas x colunas)**

```
# Dimensão dos Dados
df.shape
```

```
(5, 5)
```

```
import pandas as pd

# Caminho do arquivo CSV (ajuste se estiver em outra pasta)
caminho_csv = "Dados.csv"

# Lê o CSV e cria o DataFrame
df = pd.read_csv(caminho_csv, encoding="utf-8")
```

Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	35	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Inspeção Inicial dos Dados

- Inicialmente, verificamos algumas informações básicas dos dados.
- Para tal, após carregarmos os dados, usamos atributos e métodos da biblioteca **pandas**.

→ **Tipos de Dados**

```
# Tipos de Dados
df.dtypes
```

	0
Nome	object
Idade	int64
Peso	float64
Número de Filhos	int64
Cor dos Olhos	object

dtype: object

```
import pandas as pd

# Caminho do arquivo CSV (ajuste se estiver em outra pasta)
caminho_csv = "Dados.csv"

# Lê o CSV e cria o DataFrame
df = pd.read_csv(caminho_csv, encoding="utf-8")
```

Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	35	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Inspeção Inicial dos Dados

- Inicialmente, verificamos algumas informações básicas dos dados.
- Para tal, após carregarmos os dados, usamos atributos e métodos da biblioteca **pandas**.

→ Estatísticas Básicas

```
# Estatísticas dos Dados
df.describe()
```

	Idade	Peso	Número de Filhos
count	5.000000	5.000000	5.00000
mean	35.800000	72.440000	1.20000
std	11.344602	8.280278	1.30384
min	23.000000	62.400000	0.00000
25%	28.000000	68.900000	0.00000
50%	35.000000	70.000000	1.00000
75%	41.000000	76.800000	2.00000
max	52.000000	84.100000	3.00000

```
import pandas as pd

# Caminho do arquivo CSV (ajuste se estiver em outra pasta)
caminho_csv = "Dados.csv"

# Lê o CSV e cria o DataFrame
df = pd.read_csv(caminho_csv, encoding="utf-8")
```

Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	35	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Inspeção Inicial dos Dados

- Inicialmente, verificamos algumas informações básicas dos dados.
- Para tal, após carregarmos os dados, usamos atributos e métodos da biblioteca **pandas**.

→ **Dimensão dos Dados (linhas x colunas)**

`df.shape`

→ **Tipos de Dados**

`df.dtypes`

→ **Estatísticas Básicas**

`df.describe()`

```python
import pandas as pd

# Caminho do arquivo CSV (ajuste se estiver em outra pasta)
caminho_csv = "Dados.csv"

# Lê o CSV e cria o DataFrame
df = pd.read_csv(caminho_csv, encoding="utf-8")
```

Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	35	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Limpeza dos Dados

Alguns problemas comuns que podemos encontrar nos dados são:

- Valores irrelevantes
- Duplicatas
- Valores faltantes
- Tipos inconsistentes (ex.: número como texto)
- *Outliers*



Análise Exploratória de Dados (EDA)

Removendo Valores Irrelevantes

- Remover colunas que são irrelevantes para o objetivo do problema.

```
df.drop(['Column 1', 'Column 2', 'Column N'], axis=1)
```

- **Exemplo de Colunas Irrelevantes:** IDs, *timestamps*, coluna com valores constantes, coluna com muitos valores faltantes, colunas redundantes, etc.

```
df.drop(['Nome'], axis = 1)
```

	Idade	Peso	Número de Filhos	Cor dos Olhos
0	28	62.4	0	Castanhos
1	41	84.1	2	Verdes
2	35	70.0	1	Azuis
3	23	76.8	0	Pretos
4	52	68.9	3	Castanhos

Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	35	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Removendo Valores Irrelevantes

Eliminar colunas irrelevantes em um DataFrame é importante porque elas **atrapalham a análise e os modelos**, sem trazer informação útil. **Os principais motivos:**

- **Melhorar a Qualidade do Modelo**: Colunas sem relação com o objetivo (ex.: ID aleatório, comentários soltos) podem adicionar "ruído" e piorar o desempenho.
- **Reduz o Risco de Overfitting**: Quanto mais variáveis desnecessárias, maior a chance do modelo "decorar" padrões que não generalizam.
- **Acelera o Processamento**: Menos Colunas = Menos Memória, menos tempo para treinar modelos e rodar análises.
- **Facilita a Interpretação dos Dados**: Mais fácil entender relações, gerar gráficos e explicar resultados.
- **Evita Erros e Inconsistências**: Colunas irrelevantes muitas vezes têm formatos bagunçados (texto livre, códigos) e podem causar problemas em limpeza, normalização e conversões.
- **Ajuda na Privacidade e Governança**: Remover campos sem necessidade (ex.: CPF, telefone, e-mail) diminui exposição de dados sensíveis.

Análise Exploratória de Dados (EDA)

Renomeando Colunas

- **O nome das colunas precisa ser representativo, então em muitos casos pode ser necessário renomear para garantir a legibilidade dos dados :**

```
df.rename(columns = {nome_coluna : nome_novo})
```

```
df.rename(columns={"Idade": "Idade_anos",
                   "Peso": "Peso_kg",
                   "Número de Filhos":
                   "N_Filhos",
                   "Cor dos Olhos": "Cor_Olhos"})

df.head()
```

	Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
0	Ana Souza	28	62.4	0	Castanhos
1	Bruno Lima	41	84.1	2	Verdes
2	Carla Mendes	35	70.0	1	Azuis
3	Diego Pereira	23	76.8	0	Pretos
4	Elisa Rocha	52	68.9	3	Castanhos

Análise Exploratória de Dados (EDA)

Removendo Duplicatas

- **Determinar o Número de Linhas Duplicadas:**

```
df.duplicated().sum()
```

- **Remover as Linhas Duplicadas:**

```
df.drop_duplicates(inplace=True)
```

OBS.: "inplace=True" faz a alteração diretamente no próprio df, sem criar um novo DataFrame.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ **Quantos valores estão faltando?**

```
df.isnull().sum()
```

Como resolver esse problema?

- Remover Linhas ou Colunas Faltantes.
- Preencher o Dado Faltante com Estatísticas como Média, Moda e Mediana.



Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→**Remover Linhas ou Colunas:**

```
df.dropna()
```

```
df.dropna(inplace=True)
```

↳ Altera o df original

	Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
0	Ana Souza	NaN	62.4	0	Castanhos
1	Bruno Lima	41.0	84.1	2	Verdes
2	Carla Mendes	35.0	70.0	1	Azuis
3	Diego Pereira	23.0	NaN	0	Pretos
4	Elisa Rocha	52.0	68.9	3	Castanhos

```
df.isnull().sum()
```

	0
Nome	0
Idade	1
Peso	1
Número de Filhos	0
Cor dos Olhos	0

dtype: int64

```
df.dropna()
```

	Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
1	Bruno Lima	41.0	84.1	2	Verdes
2	Carla Mendes	35.0	70.0	1	Azuis
4	Elisa Rocha	52.0	68.9	3	Castanhos
5	Ana Souza	28.0	62.4	0	Castanhos
8	Diego Pereira	23.0	76.8	0	Pretos

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

→ **Média**: É a soma de todos os valores dividida pela quantidade total de elementos.

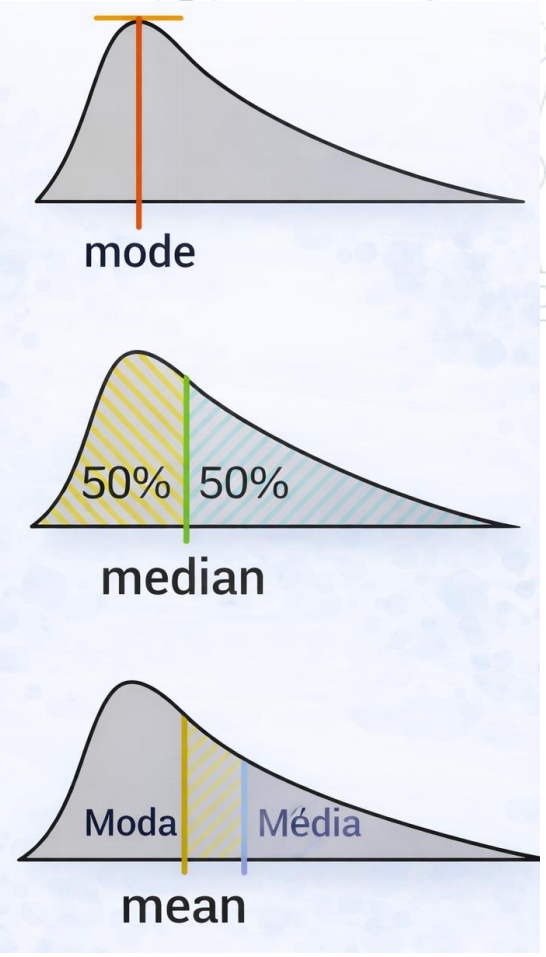
- ✓ Sensível à *outliers*.
- ✓ Usada com variáveis numéricas.

→ **Mediana**: É o valor que ocupa a posição central de um conjunto de dados ordenado. Divide os dados em 50% acima e 50% abaixo.

- ✓ Insensível a *outliers.*
- ✓ Usada com variáveis numéricas.

→ **Moda**: É o valor que aparece com a maior frequência no *dataset*.

- ✓ Insensível a *outliers.*
- ✓ Usada com variáveis numéricas e categóricas.



OBS.: Técnica usada quando o número de linhas a serem removidas é muito grande em relação ao tamanho do dataset.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

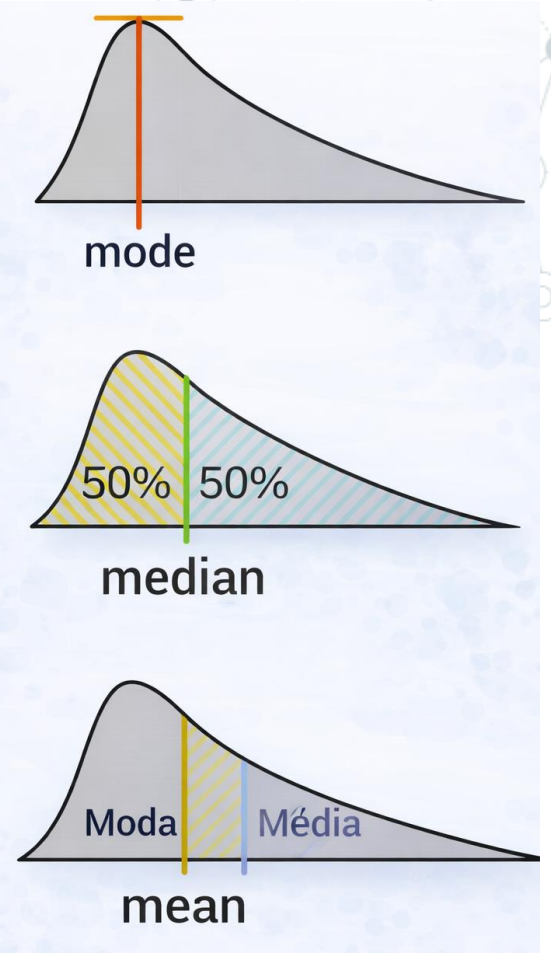
→ **Média**: É a soma de todos os valores dividida pela quantidade total de elementos.

```
num_cols = df.select_dtypes(include="number").columns
```

→ Seleciona apenas as colunas numéricas do DataFrame (Ex.: int64, float64, etc.).
→ Retorna uma lista com os nomes das colunas numéricas.

```
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
```

→ Pega todas as colunas numéricas e preenche os valores faltantes de cada uma com a sua própria média.



Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

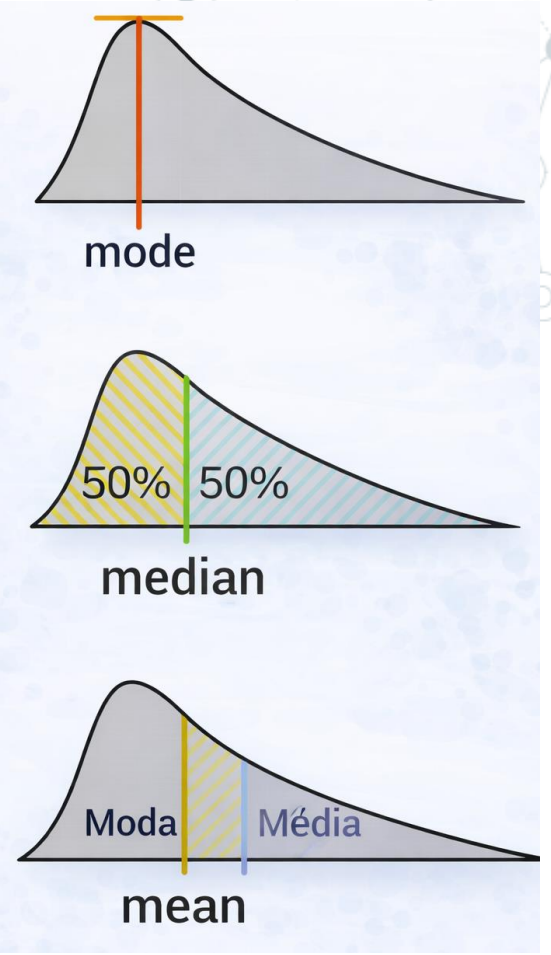
→ **Mediana**: É o valor que ocupa a posição central de um conjunto de dados ordenado. Divide os dados em 50% acima e 50% abaixo.

```
num_cols = df.select_dtypes(include="number").columns
```

→ Seleciona apenas as colunas numéricas do DataFrame (Ex.: int64, float64, etc.).
→ Retorna uma lista com os nomes das colunas numéricas.

```
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

→ Pega todas as colunas numéricas e preenche os valores faltantes de cada uma com a sua própria mediana.



Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

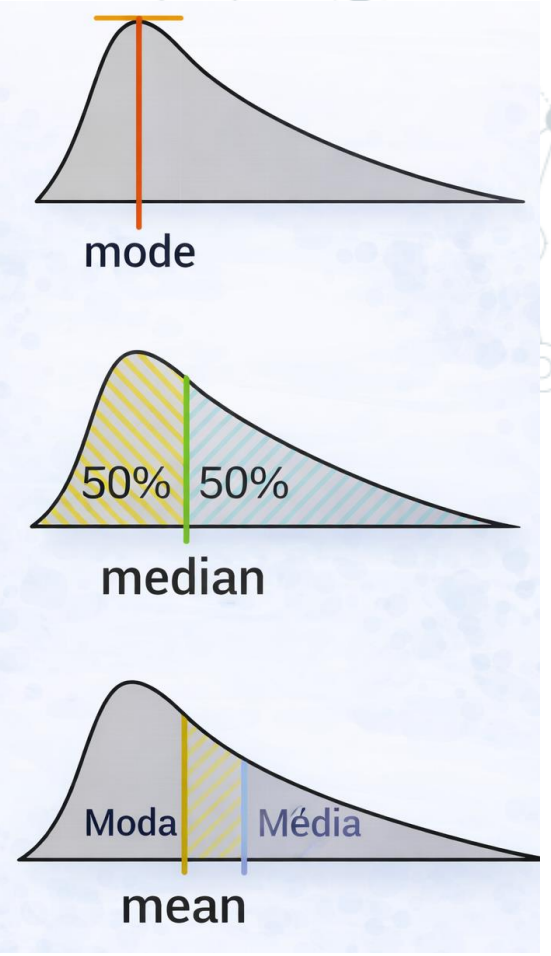
→ **Moda**: É o valor que aparece com a maior frequência no *dataset*.

```
num_cols = df.select_dtypes(include="number").columns
```

→ Seleciona apenas as colunas numéricas do DataFrame (tipos como int64, float64, etc.).
→ Retorna uma lista com os nomes das colunas numéricas.

```
df[num_cols] = df[num_cols].fillna(df[num_cols].mode().iloc[0])
```

→ Pega todas as colunas numéricas e preenche os valores faltantes de cada uma com a sua própria moda.



Observações:

- ✓ A moda pode ter mais de um valor. Assim, em geral, usamos o primeiro valor (i.e., `iloc[0]`).
- ✓ Note que a moda pode ser usada para preencher valores faltantes de variáveis numéricas ou categóricas. Portanto, para selecionar colunas categóricas, troque `include="number"` por `include="object"`.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

- Valores faltantes aparecem quando uma célula do DataFrame está vazia (NaN).
- Em vez de "chutar" um valor fixo (como 0 ou a média global), o **preenchimento inteligente** (imputação) tenta estimar o valor faltante usando **padrões presentes nos dados**, ou seja, usando outras colunas e/ou linhas.

→ Exemplo:

- ✓ Cada linha é uma "pessoa/amostra"
- ✓ Cada coluna é um "atributo/variável"
- ✓ Se um atributo está faltando, podemos estimá-lo olhando para amostras parecidas ou para relações entre variáveis

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

Pode-se usar as classes da biblioteca SciKit-Learn → **KNNImputer (K-Nearest Neighbors Imputer)**

Funcionamento:

- Para uma linha que tem um valor faltante, o método procura as **K linhas mais parecidas** (mais "próximas"), usando as colunas disponíveis.
- Depois, o valor faltante é preenchido pela **média** (ou outra agregação) do atributo nas K linhas vizinhas.

Exemplo:

✓ Se falta o **Peso** de uma pessoa, o KNNImputer busca pessoas com **Idade e Nº de Filhos parecidos** e usa os pesos delas para estimar.

- Bom quando há **grupos naturais** nos dados (pessoas semelhantes têm valores semelhantes).
- Captura padrões locais (não "achata" tudo como uma média global).

- Sensível à **escala** das variáveis: é comum precisar **padronizar** (StandardScaler) antes.
- Pode ser mais lento em bases grandes.
- Se houver muitos faltantes, fica difícil definir "vizinhos" confiáveis.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

Pode-se usar as classes da biblioteca SciKit-Learn → **IterativeImputer (imputação iterativa / tipo MICE)**

Funcionamento:

- Escolhe uma coluna com valores faltantes (ex.: Peso).
- Treina um **modelo de regressão** para prever Peso usando as **outras colunas** como entrada (features).
- Preenche os valores faltantes dessa coluna com as previsões do modelo.
- Repete isso para outras colunas, em ciclos, refinando as estimativas.

Exemplo:

✓ Prever **Peso** a partir de **Idade** e **Número de Filhos** (modelo aprende uma relação).

✓ Depois prever **Idade** a partir de Peso e outras colunas, e assim por diante.

- Aproveita **relações entre variáveis** (e não só "vizinhos").
- Pode ser bem mais preciso se existirem relações claras.

- Depende do modelo escolhido (linear, árvores, etc.) e pode introduzir viés.
- Pode ser mais complexo de configurar e mais custoso.
- Se as relações forem fracas ou os dados forem ruidosos, o ganho pode ser pequeno.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Substituição do Dado Faltante:

KNNImputer e IterativeImputer → Funcionam apenas para dados numéricos:

→ KNNImputer determina a média dos vizinhos e IterativeImputer utiliza uma técnica de regressão.

Operações que resultam em dados numéricos!

Para **variáveis categóricas** (ex.: "Cor dos Olhos"), você precisa de outras estratégias:

- Imputar pela **moda** (mais frequente),
- Usar um **modelo de classificação**,
- Aplicar técnicas específicas (ex.: imputação por grupo, "categoria desconhecida", etc.).

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Exemplo KNNImputer:

```
from sklearn.impute import KNNImputer

df_num = df.select_dtypes(include="number")

imputer = KNNImputer(n_neighbors=5) # número de vizinhos: 5

df_num_imputed = imputer.fit_transform(df_num)
```

Cria um novo DataFrame (`df_num`) contendo apenas as colunas numéricas do df.

Cria o "imputador" configurando `n_neighbors=5`.

→ Para preencher um NaN em uma linha, ele procura as 5 linhas mais parecidas (mais próximas) e usa os valores delas para estimar o faltante.

Por padrão, ele preenche usando a média do atributo (coluna) nesses 5 vizinhos.

O método '`fit_transform`' treina o modelo e preenche os valores faltantes

Observação:

- Em ML, calculamos o valor faltante **usando apenas os dados do conjunto de treinamento** (para evitar *data leakage*). Isso é válido para qualquer pré-processamento/transformação de dados.

Análise Exploratória de Dados (EDA)

Tratando Valores Faltantes

→ Exemplo IterativeImputer:

```
from sklearn.experimental import enable_iterative_imputer

from sklearn.impute import IterativeImputer

df_num = df.select_dtypes(include="number")

imputer = IterativeImputer()

df_num_imputed = imputer.fit_transform(df_num)
```

Cria um novo DataFrame (`df_num`) contendo apenas as colunas numéricas do df.

Cria o imputador com parâmetros padrão.

→ Por padrão, ele usa um modelo do tipo BayesianRidge (regressão) para prever valores faltantes.

→ A lógica é: cada coluna com NaN vira, por vez, a "variável alvo" e é prevista usando as outras colunas como entrada.

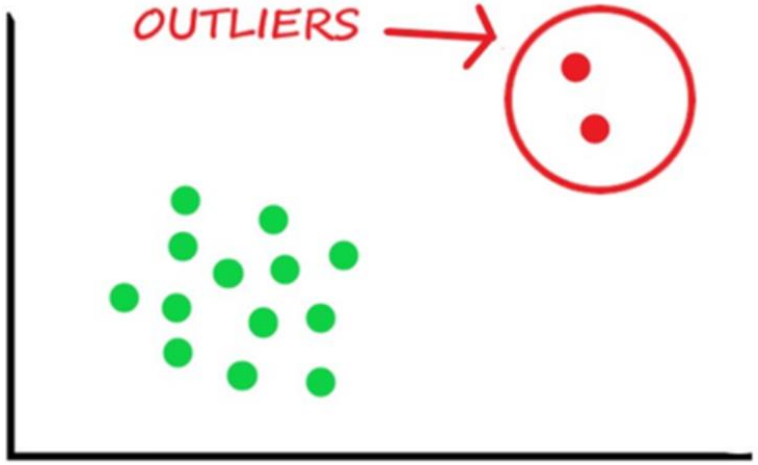
O método '`fit_transform`' treina o modelo e preenche os valores faltantes

Análise Exploratória de Dados (EDA)

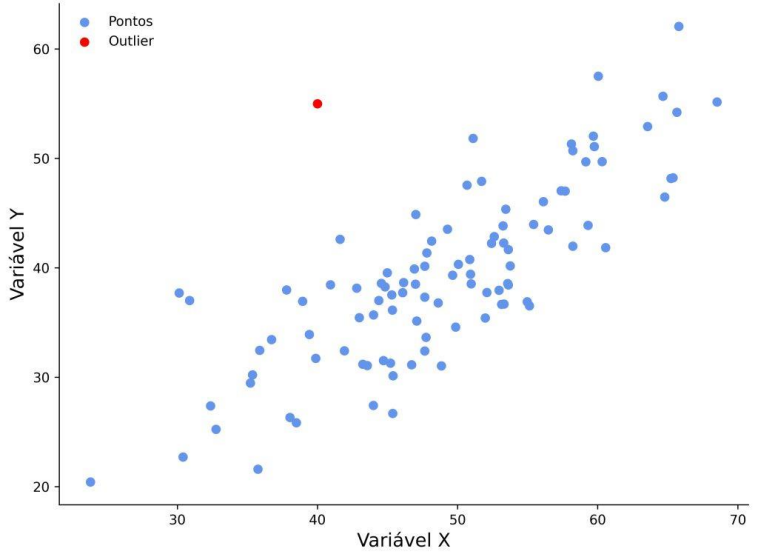
Detecção de Outliers

O que é um Outlier?

Um **outlier** (valor atípico) é uma observação que fica **muito distante da maioria dos dados**, parecendo "fora do padrão" do conjunto.



Nome	Idade	Peso	Número de Filhos	Cor dos Olhos
Ana Souza	28	62.4	0	Castanhos
Bruno Lima	41	84.1	2	Verdes
Carla Mendes	250	70.0	1	Azuis
Diego Pereira	23	76.8	0	Pretos
Elisa Rocha	52	68.9	3	Castanhos

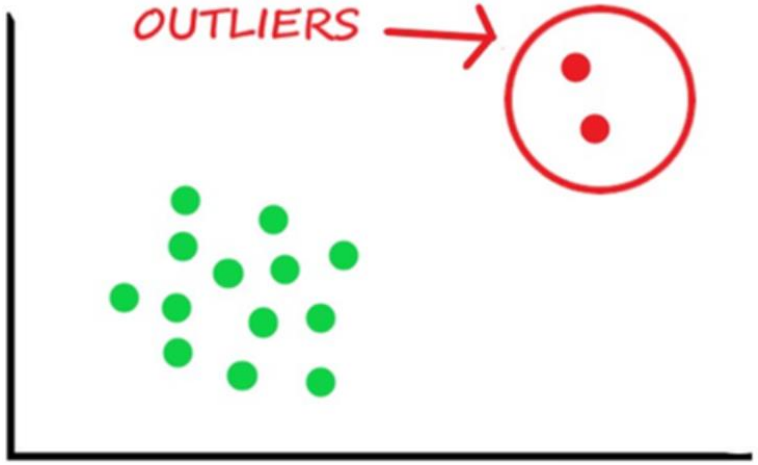


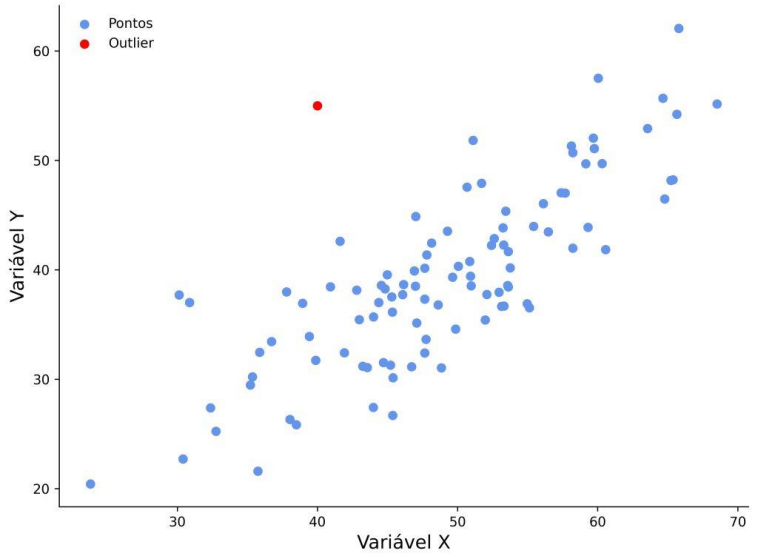
Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ **Um *outlier* pode ser:**

- ✓ Um erro de medição ou registro (e.g., digitação),
- ✓ Erro de processamento dos dados (e.g., conversão errada de unidade)
- ✓ Um valor raro, mas válido (e.g., uma fraude),
- ✓ Uma cauda longa (distribuição assimétrica),
- ✓ Valor de um grupo/classe diferente.



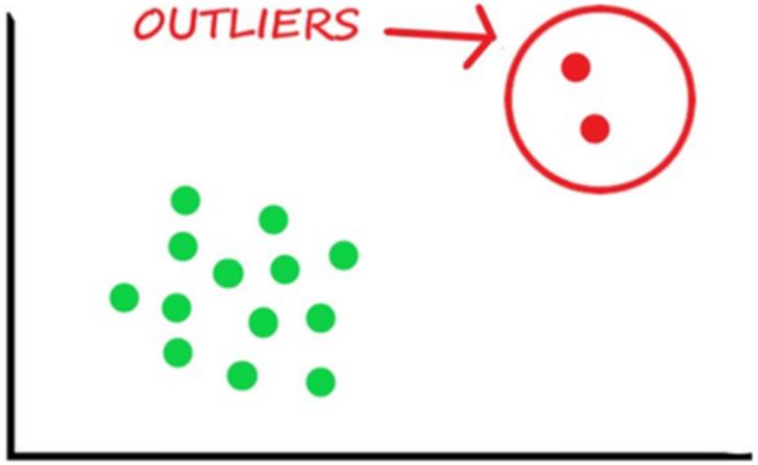


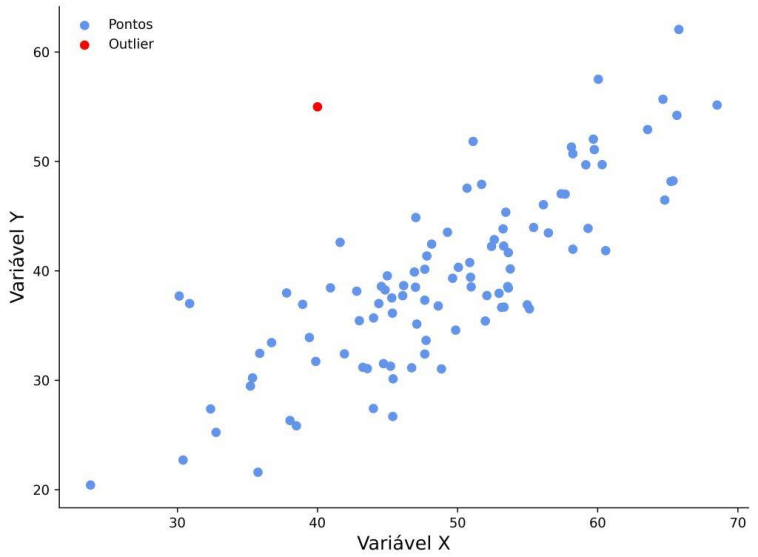
Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ **Por que importam?**

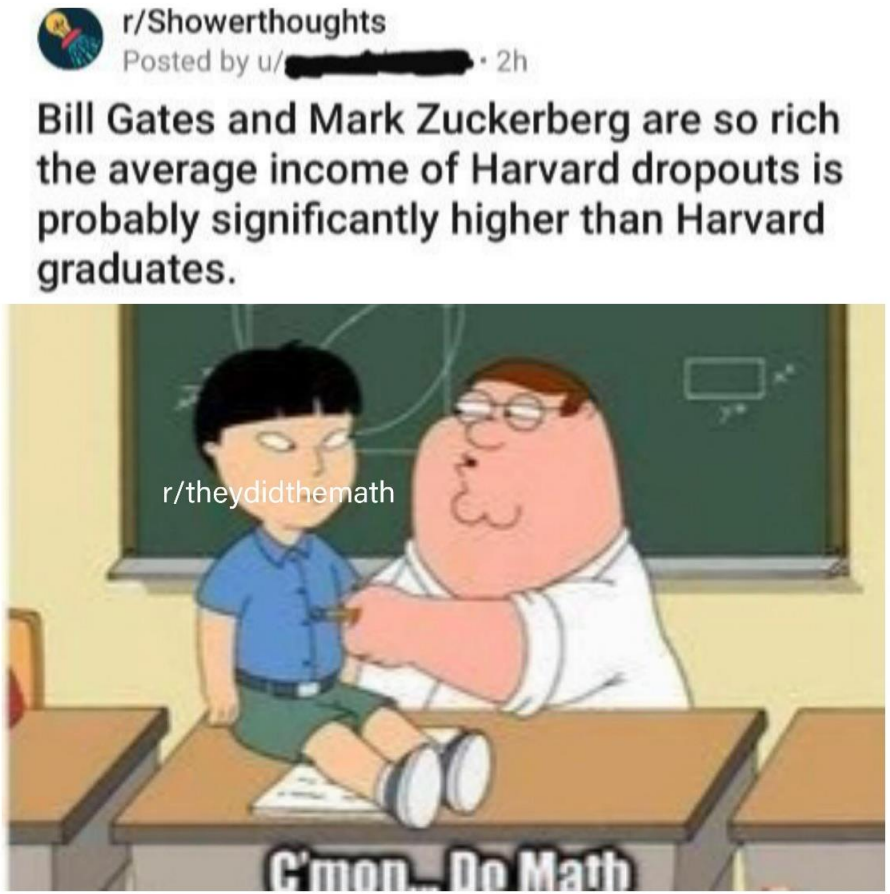
- ✓ **Distorcem estatísticas:** Afetam média, desvio padrão e correlações.
- ✓ **Prejudicam visualizações:** Esticam a escala e escondem padrões reais.
- ✓ **Impactam modelos de ML**, especialmente os baseados em distância
- ✓ **Geram modelos enviesados e de pior desempenho.**
- ✓ **Podem revelar informação importante:** Outliers podem revelar evento raro relevante como fraude, falha em sistemas, ataques cibernéticos.





Análise Exploratória de Dados (EDA)

Detecção de Outliers



- O post diz que a média salarial de quem largou Harvard é maior que a de quem se formou.
- Porém, largar a faculdade, provavelmente, não deixará vocês bilionários.
- O dado está correto, mas ele é uma **exceção** (não um erro) tão grande que destrói a representatividade da média.

Análise Exploratória de Dados (EDA)

Detecção de Outliers



Se estivéssemos medindo a temperatura de um servidor e um sensor marcasse 500°C por um segundo, isso é um erro ou uma exceção que deve ser investigada?

- *Outliers* são exceções.
- Porém, exceções não são necessariamente erros.
- Nunca devemos removê-los ou ignorá-los sem investigar.

Análise Exploratória de Dados (EDA)

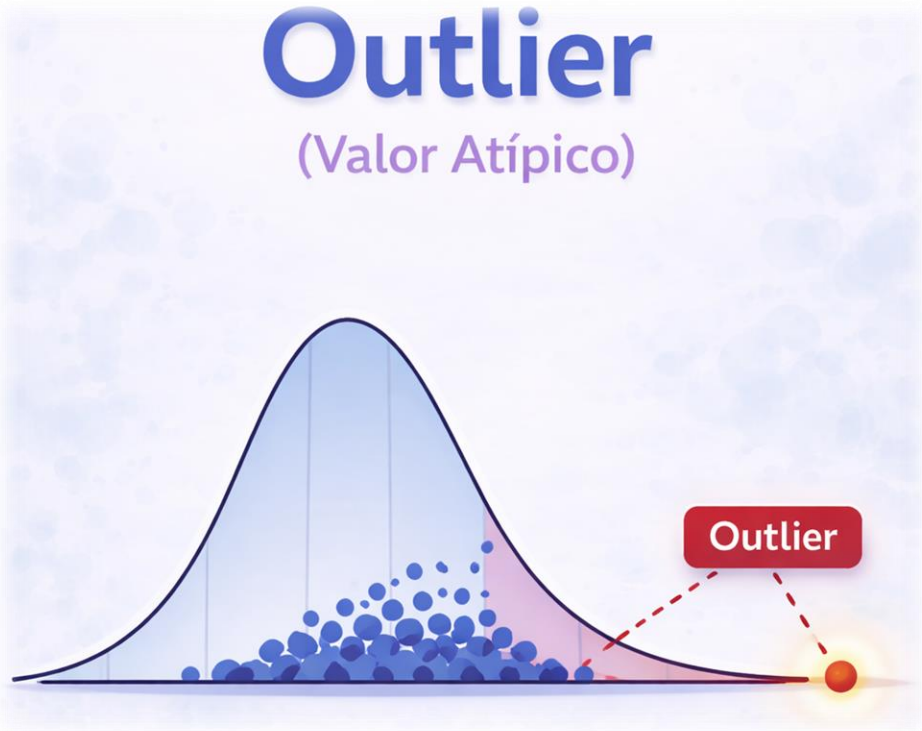
Detecção de Outliers

De forma visual:

- ✓ Histograma
- ✓ Boxplot

De forma estatística:

- ✓ Z-score
- ✓ IQR (intervalo interquartil)



Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ HISTOGRAMA:

- **Pandas →** `df['Price'].hist(bins=30)`

O parâmetro `bins` ajusta o tamanho das caixas de contagem e é opcional, por padrão, é igual a 10. Funciona bem com valores de 20 a 30.

- **Matplotlib (mais controle):**

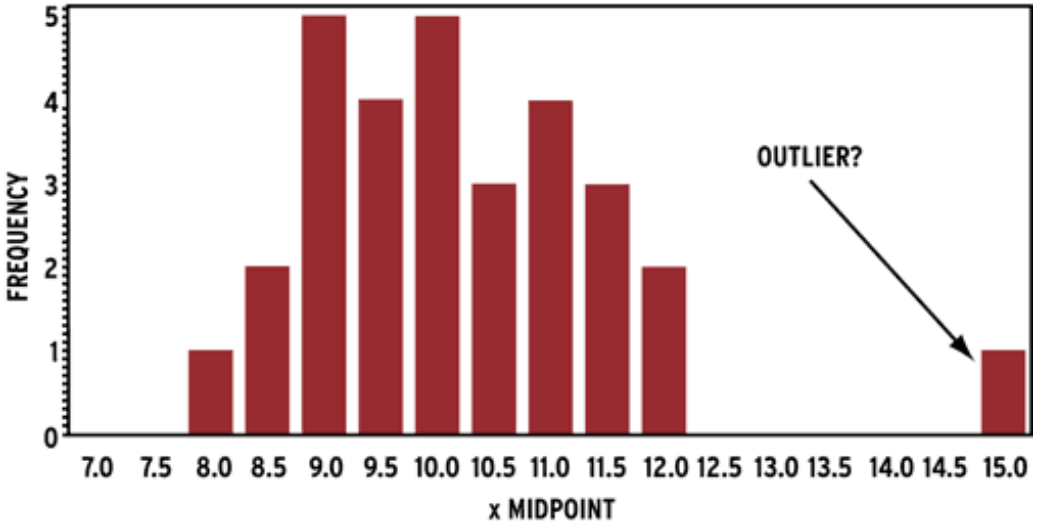
```
plt.hist(df['Price'], bins=30, density=True)
plt.xlabel('Preço')
plt.ylabel('Frequência')
plt.title('Histograma do Preço')
plt.show()
```

Os parâmetros `density` e `cumulative` estimam a PDF e a CDF (padrão, `False`).

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ HISTOGRAMA:



→ Histogramas ajudam, mas não são precisos sozinhos.

- Dependem do número de bins.
- Não definem limites objetivos.
- Um outlier pode "sumir" em um bin largo.

→ Eles são ótimos para uma visão geral, sugerem a presença, mas não devem ser usados para a decisão final.

Então, o que usar para detectar outliers?

Análise Exploratória de Dados (EDA)

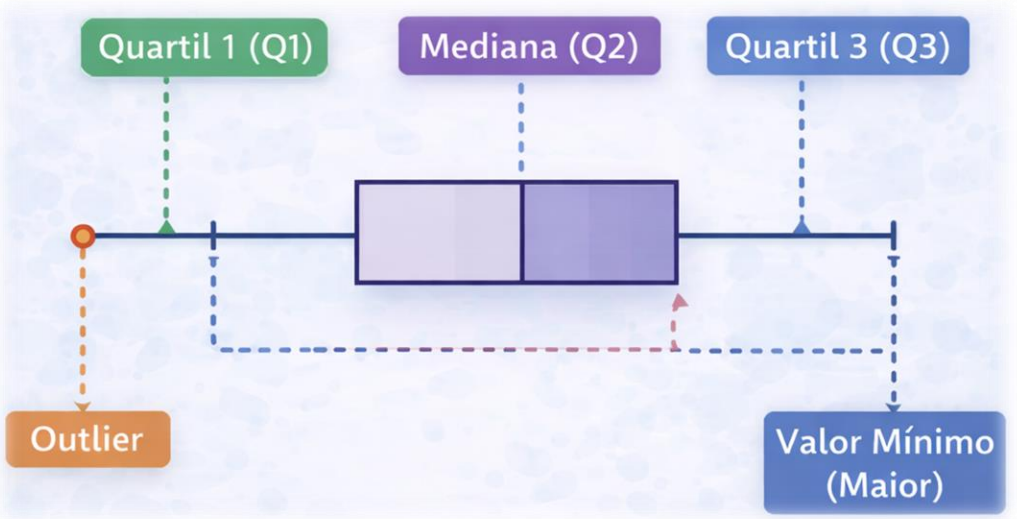
Detecção de Outliers

→ BOXPLOT:

- **Pandas →** `df['Year'].plot.box()`

- **Com Matplotlib (mais controle):**

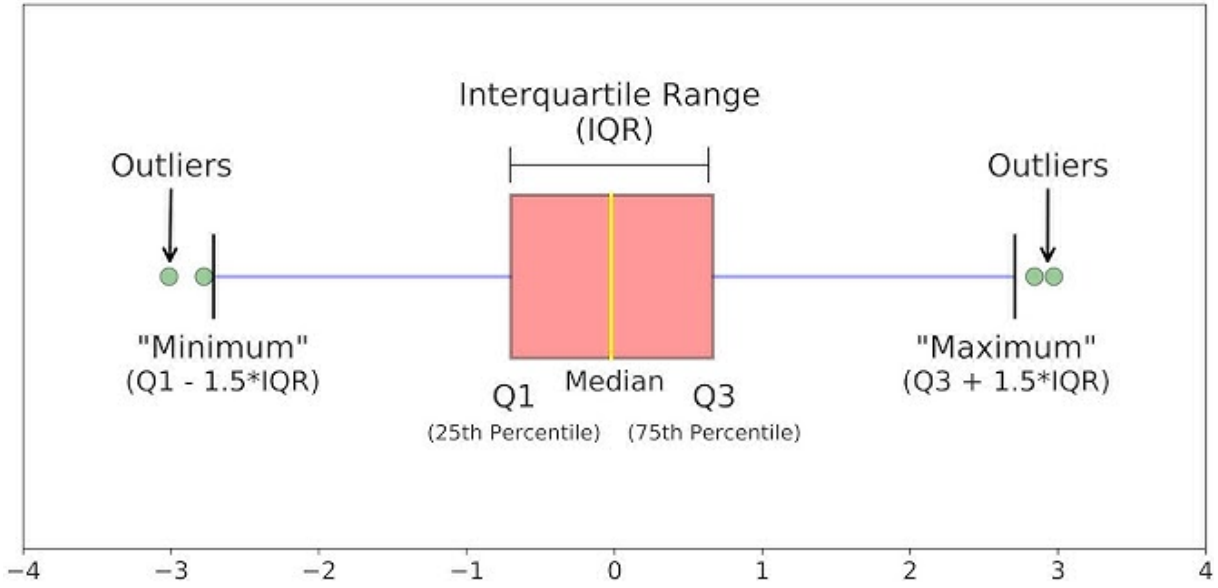
```
plt.boxplot(df['Year'])
plt.xlabel('Year')
plt.ylabel('Valores')
plt.title('Boxplot do Year')
plt.show()
```



Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ BOXPLOT:



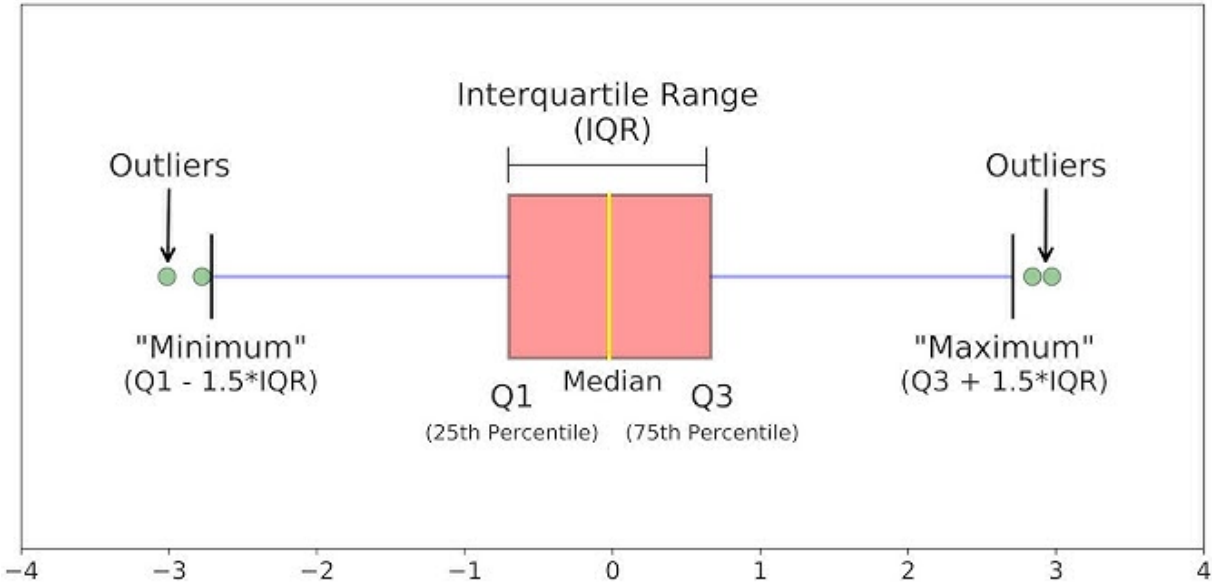
- **Q1 (1º quartil / 25º percentil):** ponto abaixo do qual estão 25% dos dados.
- **Mediana (Q2 / 50º percentil):** divide os dados ao meio (50% abaixo e 50% acima).
- **Q3 (3º quartil / 75º percentil):** ponto abaixo do qual estão 75% dos dados.
- **Caixa (box):** vai de Q1 até Q3 e contém o "miolo" dos dados (os 50% centrais).

- **Borda inferior da caixa** → primeiro quartil (Q1): 25% dos dados estão abaixo desse valor.
- **Linha dentro da caixa** → mediana (Q2): valor central dos dados. Metade dos valores está acima e metade abaixo.
- **Borda superior da caixa** → terceiro quartil (Q3): 75% dos dados estão abaixo desse valor.

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ BOXPLOT:



- **Q1 (1º quartil / 25º percentil):** ponto abaixo do qual estão 25% dos dados.
- **Mediana (Q2 / 50º percentil):** divide os dados ao meio (50% abaixo e 50% acima).
- **Q3 (3º quartil / 75º percentil):** ponto abaixo do qual estão 75% dos dados.
- **Caixa (box):** vai de Q1 até Q3 e contém o "miolo" dos dados (os 50% centrais).

- **Caixa** → intervalo interquartil (IQR = Q3-Q1): mostra a variabilidade dos dados em torno da mediana.
  - Se a mediana estiver deslocada dentro da caixa, indica **assimetria** na distribuição.
- **Linhas que saem da caixa** → bigodes (*whiskers*): vão até o menor e maior valores que não são *outliers*.
  - Os limites são geralmente definidos por:
    Q1 − 1.5×IQR
    Q3 + 1.5×IQR
- **Pontos fora dos bigodes** → possíveis *outliers*: são os valores abaixo e acima dos intervalos anteriores.

ANÁLISE EXPLORATÓRIA DE DADOS (EDA)

Detecção de Outliers

→ Z-SCORE:

```
num_cols = df.select_dtypes(include='number').columns
z_scores = np.abs(stats.zscore(df[num_cols]))
outliers_por_linha = (z_scores > 3).any(axis=1)
df_outliers = df[outliers_por_linha]
```

Cria um novo DataFrame (`df_num`) contendo apenas as colunas numéricas do df.

Calcula o **z-score** para cada valor das colunas numéricas e pega o valor absoluto.

Cria uma máscara booleana indicando **quais linhas têm pelo menos um outlier**.

→ `(z_scores > 3)`: retorna True onde o z-score absoluto é maior que 3 (regra comum de outlier).

→ `.any(axis=1)`: verifica linha por linha se existe alguma coluna com True.

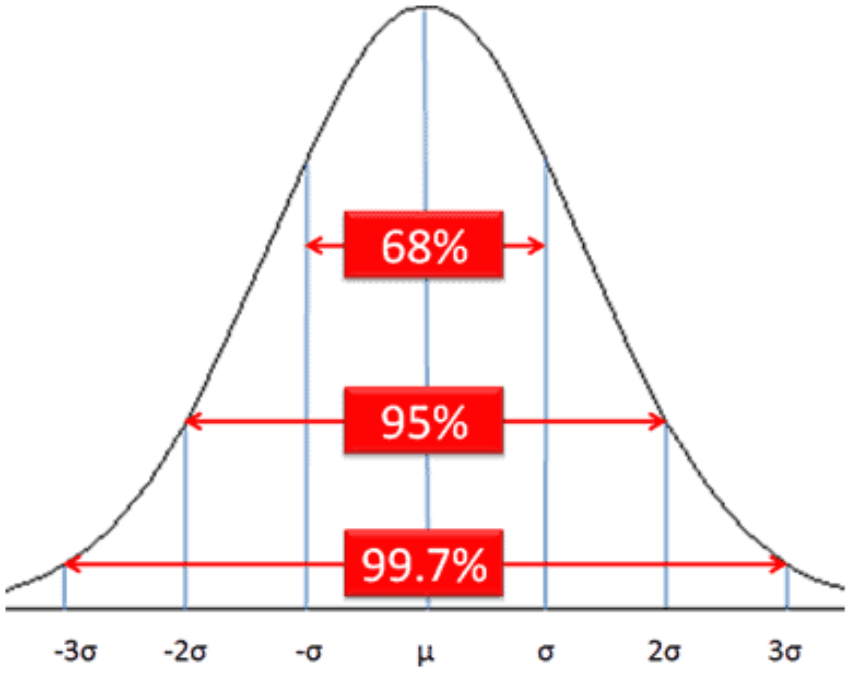
Filtra o DataFrame original df e retorna somente as linhas marcadas como outliers.

`df_outliers` é um DataFrame contendo os registros "suspeitos", com base no critério |z| > 3.

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ Z-SCORE:

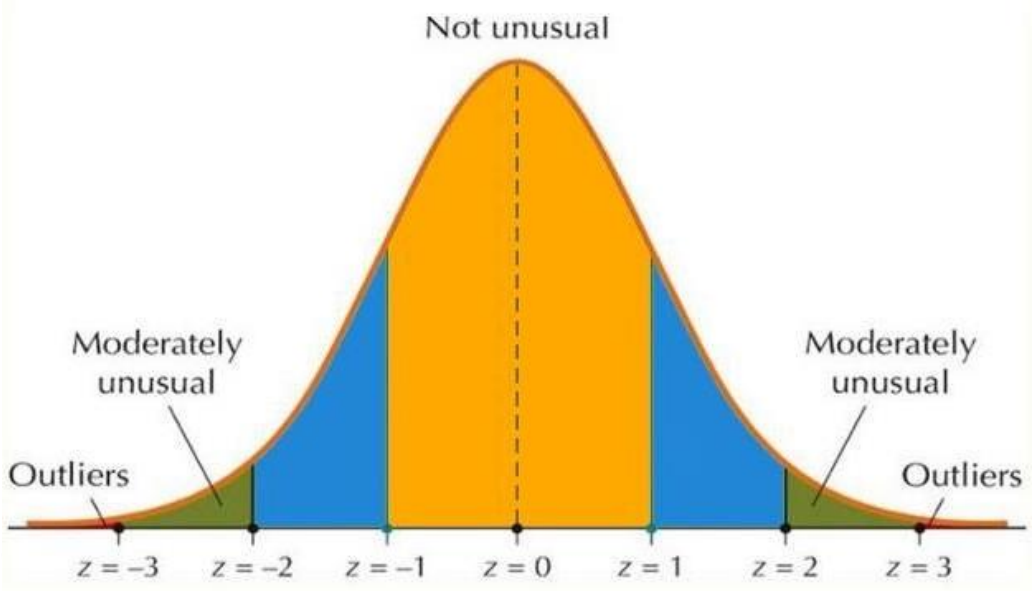


- Mede quantos desvios-padrão um valor está distante da média da distribuição.
- Assume que os dados seguem uma distribuição semelhante à normal.
- 99.7% dos valores estão dentro do intervalo de +/- 3σ.
- Assim, assume-se que valores maiores do que +/- 3σ são possíveis *outliers*, pois são valores raros.

ANÁLISE EXPLORATÓRIA DE DADOS (EDA)

Detecção de Outliers

→ Z-SCORE:



- **Z-score de um valor x é calculado como:**

$$z = \frac{x - \mu}{\sigma}$$

z indica a distância até a média:

→ $z = 0 \rightarrow$ valor igual à média

→ $z = 1 \rightarrow$ 1 desvio-padrão acima da média

→ $z = 2 \rightarrow$ 2 desvios-padrão abaixo da média

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ VANTAGENS E DESVANTAGENS --- BOXPLOT (IQR):

Vantagens

- Robusto a valores extremos, pois usa quantis e não média;
- Funciona bem mesmo com distribuições assimétricas;
- Fácil de interpretar visualmente (*boxplots*);
- Não assume nenhuma distribuição específica dos dados, i.e., independente da forma da distribuição.

Desvantagens

- Pode falhar em distribuições multimodais (vários picos);
- O fator 1.5 é empírico e pode não ser ideal em todos os contextos;
- Em conjuntos de dados grandes, pode definir muitos pontos como outliers ou não mostrar o verdadeiro intervalo dos dados em conjuntos menores;
- Ineficaz para conjuntos de dados pequenos, pois não representam a distribuição com precisão.

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ VANTAGENS E DESVANTAGENS --- Z-SCORE:

Vantagens

- Funciona bem quando os dados seguem uma distribuição aproximadamente normal;
- Permite comparar atributos em escalas diferentes (dados são padronizados);
- Simples de calcular e interpretar.

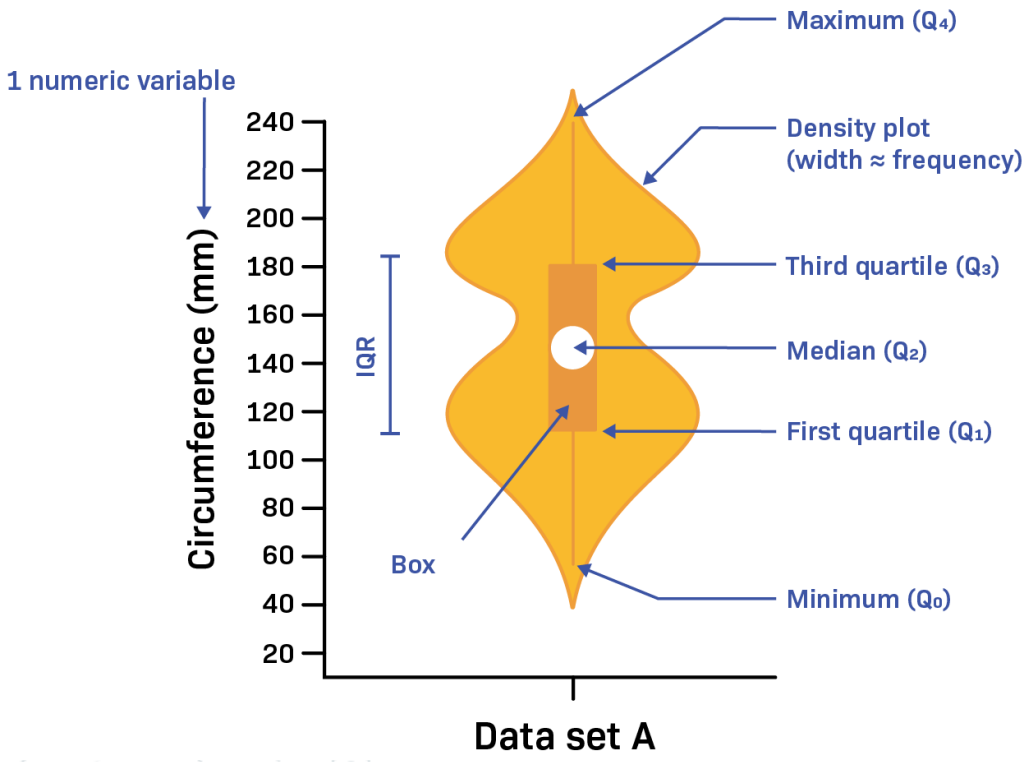
Desvantagens

- Sensível a outliers, pois média e desvio-padrão são afetados por valores extremos;
- Não funciona bem com distribuições assimétricas;
- Pode gerar muitos falsos positivos em distribuições assimétricas;
- Parâmetros são necessários: μ e σ da população devem ser conhecidos;
- Caso contrário, devem ser usadas estimativas da amostra, o que pode ser um problema com *datasets* pequenos.

Análise Exploratória de Dados (EDA)

Detecção de Outliers

→ OUTROS MÉTODOS:



Além de IQR e z-score, existem **técnicas mais avançadas** para detectar outliers, especialmente em **dados complexos, multidimensionais ou não gaussianos.**

→ **Métodos usando algortimos de ML:** Isolation Forest, Local Outlier Factor, One-Class SVM, Autoencoders.

→ **Métodos visuais:** Violin plot -- Combina *boxplot* com densidade, destacando assimetrias.

Análise Exploratória de Dados (EDA)

Relação entre Variáveis

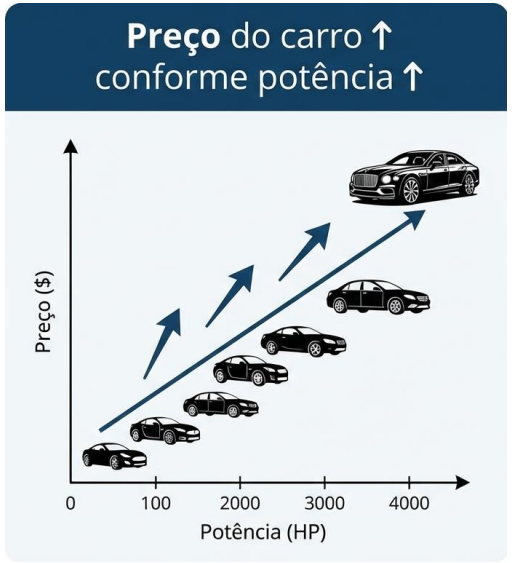


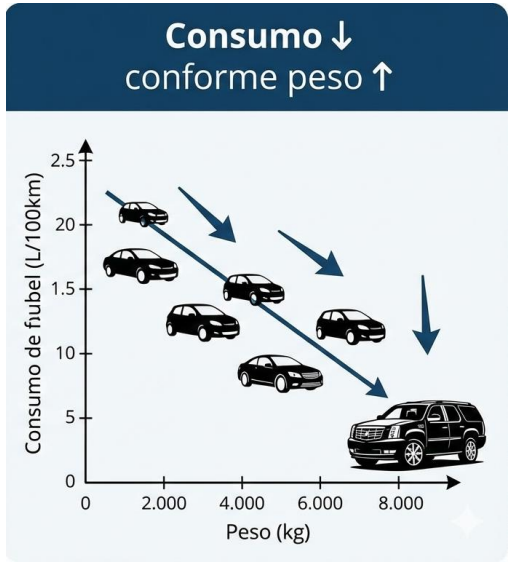


Correlação não implica causalidade!!

Análise Exploratória de Dados (EDA)

Relação entre Variáveis





Objetivo: Descobrir padrões e estruturas escondidas nos dados.

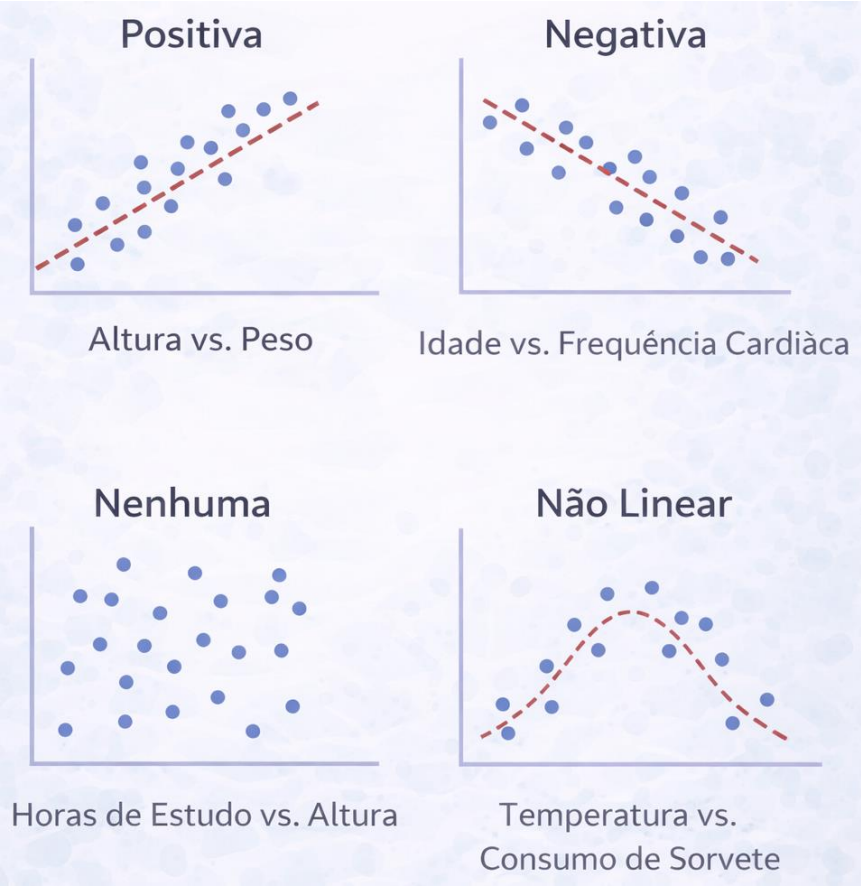
- Relações entre variáveis revelam:
- Tendências
- Dependências
- Agrupamentos naturais

Ao analisar relações, podemos responder:

- O que realmente influencia o objetivo?
- Essa variável faz sentido nesse contexto?
- Esse comportamento é esperado ou estranho?

Análise Exploratória de Dados (EDA)

Relação entre Variáveis



Correlação

Avaliar as Relações entre variáveis permite:

- **Selecionar atributos relevantes**
- **Remover atributos redundantes**
- **Escolher modelos de ML adequados**
- **Detectar multicolinearidade:** Detectar multicolinearidade é o processo de identificar quando duas ou mais variáveis explicativas (features) em um conjunto de dados estão altamente correlacionadas entre si, ou seja, carregam informação redundante, afetando negativamente a

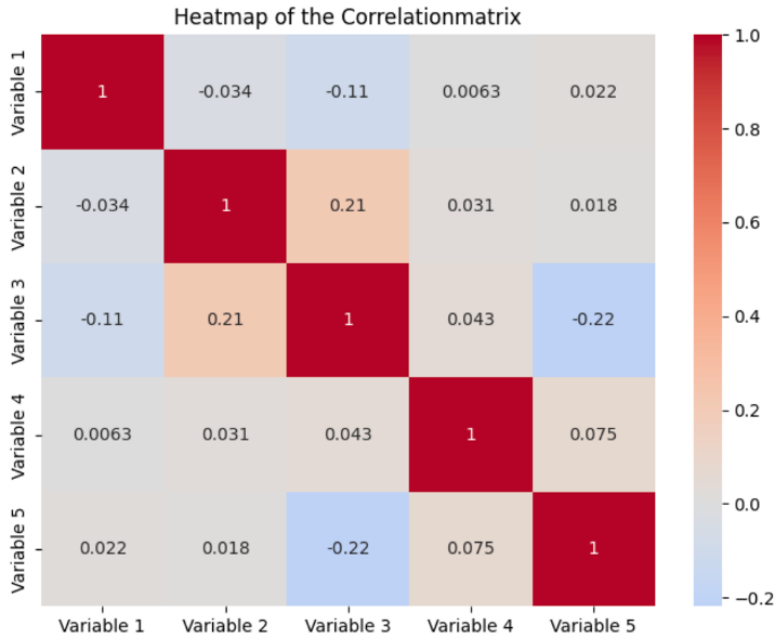
  ✓ interpretação dos coeficientes dos modelos,
  ✓ generalização do modelo, levando a modelos instáveis,
  ✓ predição dos modelos de ML.

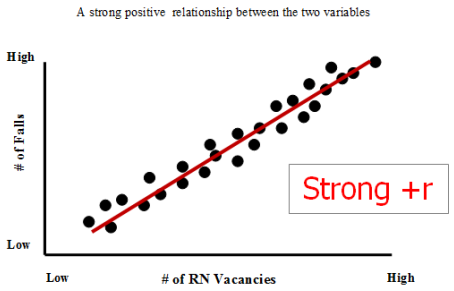
Análise Exploratória de Dados (EDA)

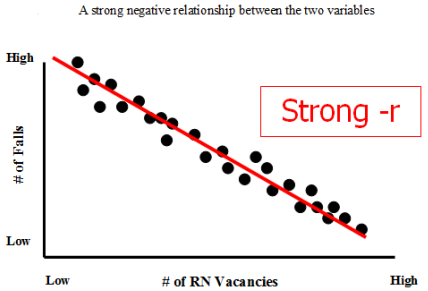
Relação entre Variáveis

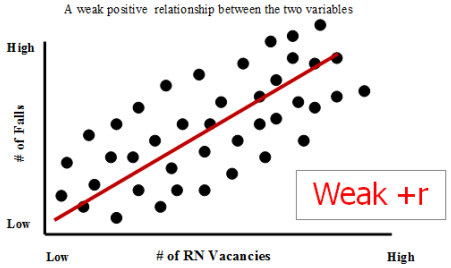
FERRAMENTAS:

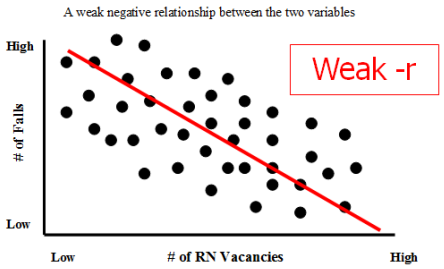
- ✓ Matriz de correlação (*heatmap*)
- ✓ Diagrama de dispersão (*scatterplot*)











Análise Exploratória de Dados (EDA)

Relação entre Variáveis

→ Matriz de Correlação (*heatmap*)

O *heatmap* mostra, de forma visual, o grau de **relação linear** entre pares de variáveis numéricas. Cada elemento da matriz é calculado usando-se o coeficiente de correlação de Pearson:

$$\rho = \frac{\sum_{i=1}^{N}(x_i - \mu_x)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^{N}(x_i - \mu_x)^2}\sqrt{\sum_{i=1}^{N}(y_i - \mu_y)^2}} \approx \frac{\operatorname{cov}(X,Y)}{\sqrt{\operatorname{var}(X)\operatorname{var}(Y)}}$$

Valores variam de −1 a +1

+1 → relação linear positiva forte: Y aumenta à medida que X aumenta.

−1 → relação linear negativa forte: Y aumenta enquanto X diminui.

0 → nenhuma **relação linear** entre X e Y.

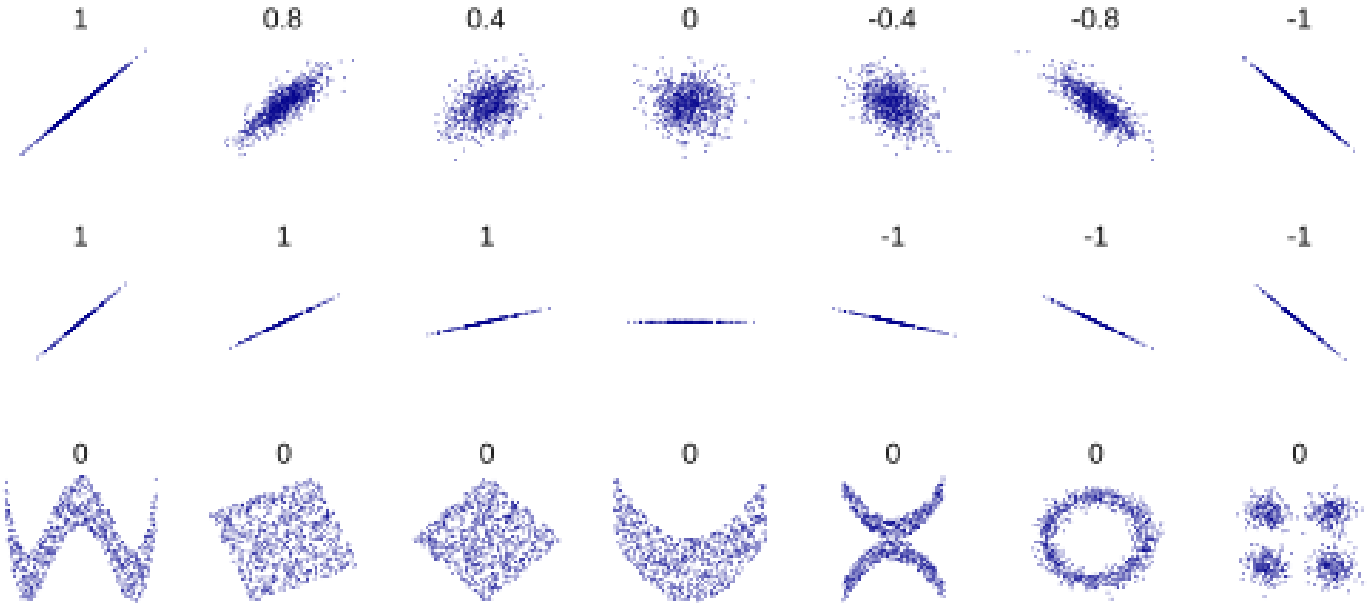
OBS.: Valores negativos são chamados de anti-correlação → **Anti-correlação** é basicamente uma **correlação negativa**: quando **uma variável aumenta, a outra tende a diminuir** de forma consistente.

Análise Exploratória de Dados (EDA)

Relação entre Variáveis

→ Correlação de Pearson

A **correlação de Pearson** mede a **força e a direção da relação linear** entre duas variáveis numéricas.



→ Se ruído aumenta, então $\rho \to 0$.

- Uma variável não fornece informação sobre a outra.

→ ρ não depende da inclinação

- Inclinações mais ou menos acentuadas resultam no mesmo ρ.

→ O coeficiente de Pearson, ρ, é 0 se a correlação é não-linear

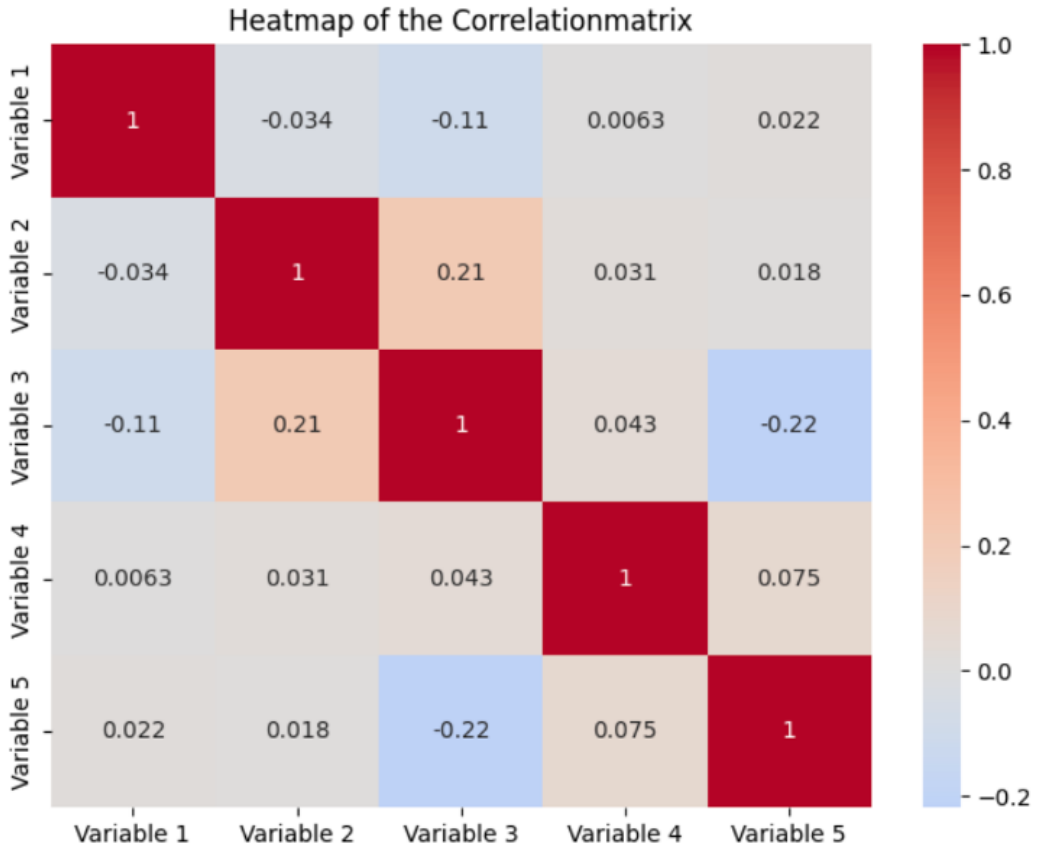
- Mesmo y sendo uma função não linear de x (e.g., $y = \sin(x)$), ρ será 0.

→ ρ é 0 se não existe relação linear.

Análise Exploratória de Dados (EDA)

Relação entre Variáveis

→ Matriz de Correlação



- Identificar variáveis fortemente (linearmente) relacionadas.
- Detectar multicolinearidade (↓ redundância).
- Apoiar seleção de atributos.
- Levantar hipóteses sobre relações entre variáveis.

Limitações:

✓ ρ é sensível a *outliers* por usar a média em seu cálculo.

✓ Não mostra a forma da relação e, portanto, não indica se a relação é não-linear → Devemos analisar visualmente (*scatter plot*) e/ou com outro coeficiente (Spearman).

Análise Exploratória de Dados (EDA)

Relação entre Variáveis

→ Correlação de Spearman

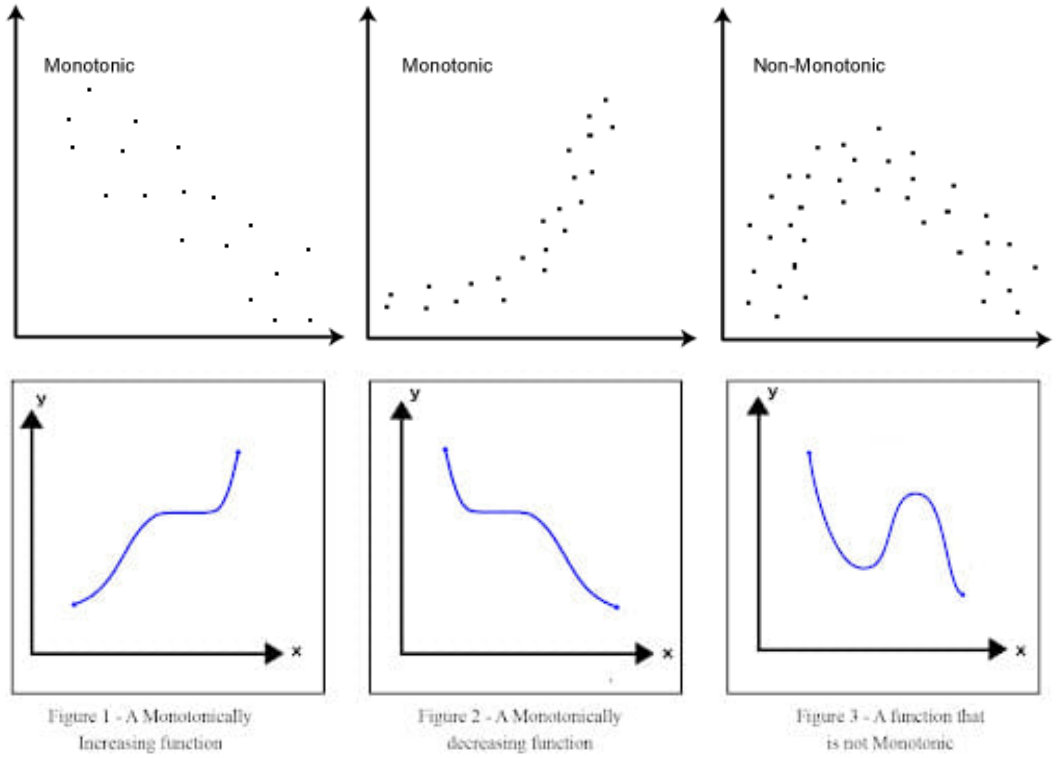


Figure 1 - A Monotonically Increasing function

Figure 2 - A Monotonically decreasing function

Figure 3 - A function that is not Monotonic

Se os dados crescem em formato de curva (exponencial, logarítmica), o Pearson dará um valor baixo, mas o Spearman dará 1.0.

- A **correlação de Spearman** (ρ, "rho") mede a **força e a direção de uma relação monotônica** entre duas variáveis, ou seja, quando uma cresce, a outra tende a crescer (ou diminuir), **mesmo que não seja uma relação linear**.
- Também avalia a relação de dados ordinais, i.e., dados que não são números exatos, mas têm uma ordem.
- Valor próximo de 0 se a relação for não-monotônica, i.e., se ela mudar de direção (e.g., $y = \sin(x)$).
- **Mais robusto** a outliers e adequado para dados não normais.
- **Limitação:** Se a relação **não é monotônica** (ex.: sobe e depois desce), Spearman pode dar perto de 0.

✓ ρ = +1: relação monotônica crescente perfeita (quanto maior X, maior Y)
✓ ρ = −1: relação monotônica decrescente perfeita (quanto maior X, menor Y)
✓ ρ ≈ 0: não há relação monotônica clara

Análise Exploratória de Dados (EDA)

→ Matriz de Correlação

Usando o coeficiente de Pearson

```
corr = df.corr(method='pearson', numeric_only=True)
plt.figure()
sns.heatmap(corr, cmap="BrBG", annot=True)
plt.show()
```

⇨ Calcula a matriz de correlação do DataFrame df.

⇨ Cria um **heatmap** (mapa de calor) usando Seaborn.

Usando o coeficiente de Spearman

```
corr = df.corr(method='spearman', numeric_only=True)
plt.figure()
sns.heatmap(corr, cmap="BrBG", annot=True)
plt.show()
```

⇨ Calcula a matriz de correlação do DataFrame df.

⇨ Cria um **heatmap** (mapa de calor) usando Seaborn.

Análise Exploratória de Dados (EDA)

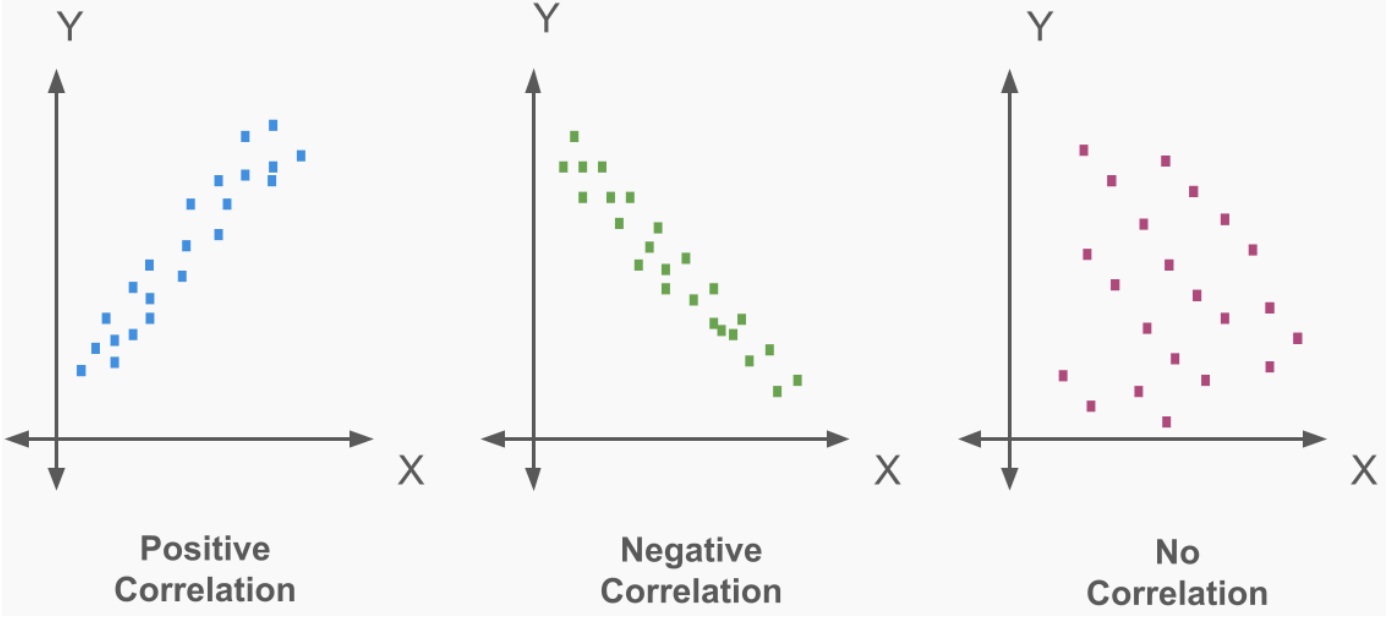
Relação entre Variáveis

→ Diagrama de Dispersão (*scatter plot*)

Mostra a relação direta entre duas variáveis, ponto a ponto, revelando padrões que números não mostram.

Permite visualizar:

- ✓ tendências (positiva ou negativa),
- ✓ relações não-lineares,
- ✓ *clusters* (i.e., grupos) e
- ✓ *Outliers*



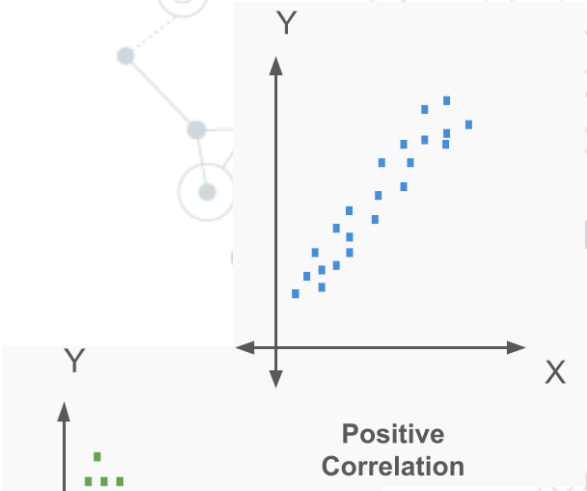
Análise Exploratória de Dados (EDA)

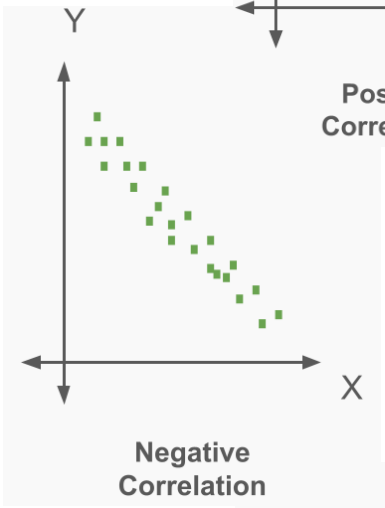
Relação entre Variáveis

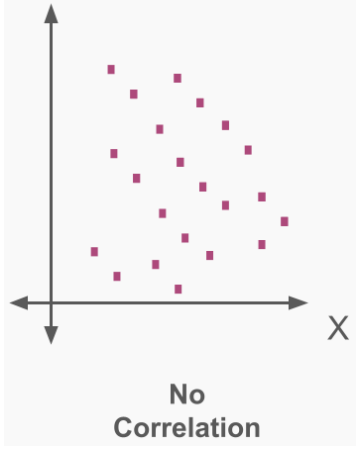
→ Diagrama de Dispersão (*scatter plot*)

Limitações:

- Funciona bem para poucas variáveis.
- Difícil de escalar para alta dimensionalidade. Quando o número de variáveis cresce:
  - → Seriam necessários muitos scatter plots
  - → A análise vira um processo manual, cansativo e propenso ao erro
  - → **Exemplo:** 20 variáveis se tornam 190 pares possíveis
- Pode ficar poluído em *datasets* grandes. Em datasets com muitos pontos:
  - → Os pontos se sobrepõem
  - → Regiões densas viram "manchas"
  - → Outliers podem ficar escondidos







Análise Exploratória de Dados (EDA)

Relação entre Variáveis

→ Diagrama de Dispersão (*scatter plot*)

Plotando a dispersão de um par de variáveis

```
fig, ax = plt.subplots()
ax.scatter(df['MPG-C'], df['Price'])
ax.set_xlabel('MPG-C')
ax.set_ylabel('Price')
ax.grid(True)
plt.show()
```

⇨ Plota um gráfico de dispersão

Plotando scatter plots para todos os pares de variáveis

```
sns.pairplot(df)
plt.show()
```

⇨ Plota uma matriz de scatter plots para todos os pares de variáveis.

Análise Exploratória de Dados (EDA)

Pandas Profiling

O Pandas profiling (ou *ydata-profiling*) é uma biblioteca que automatiza grande parte da análise exploratória de dados (EDA) gerando um relatório completo com uma única linha de código.

O relatório que inclui:

- ✓ Resumo geral dos dados: número de linhas e colunas, tipos de dados.
- ✓ Estatísticas univariadas: média, mediana, moda, desvios-padrão, histogramas.
- ✓ Identificação de problemas: valores faltantes, duplicatas e potenciais anomalias.
- ✓ Correlação e interações entre variáveis numéricas e categóricas.
- ✓ Visualizações: de distribuições, barras, densidades e matrizes de correlação.

O relatório pode ser salvo em HTML ou em um *notebook* Jupyter.

Análise Exploratória de Dados (EDA)

Pandas Profiling

Como Gerar o Relatório?

```
from ydata_profiling import ProfileReport

# Cria um objeto de relatório com base no DataFrame df.
# title="Relatório de Dados" define o título que aparecerá no relatório.

profile = ProfileReport(df, title="Relatório de Dados")

# Salva em um arquivo HTML.
profile.to_file("relatorio.html")

# Abre o relatório no notebook Jupyter.
profile.to_notebook_iframe()
```

Análise Exploratória de Dados (EDA)

EXEMPLO:

- ✓ Exemplo: intro_eda.ipynb
- ✓ **Github:** Prof. Felipe Figueiredo





Obrigada pela Atenção!

Dúvidas?

victoria.souto@Inatel.br